10 Atropos Source Doc DAG

(Android APK, Web Server, SpecGraph Vercel Production, CLI UI/UX & Universal Web)

BATCH 1 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (SECTIONS 1 TO 5)

ATOM 1.1: HARDWARE PROFILE AND PLATFORM DETECTION ABSTRACTION File Path: src/main/kotlin/atropos/core/adapter/HardwareProfileAdapter.kt Callers and Interfaces: Invoked by PlatformWire, TermuxProotShim, and SystemCommandCatalog through PlatformAdapterRegistry. Values and Symbols: HardwareProfile data class, getProfile, getArch, arch string, cpuCores integer, totalMemoryBytes long. Imports and Exports: Imports java.io.File; exports HardwareProfile immutable model to core platform runtime. Implementation Directive: Remove all hardcoded developer architecture assumptions. Read host architecture dynamically from system properties os.arch and runtime availableProcessors, normalizing to arm64, x86_64, or portable fallback without defaulting to development machine parameters. Further Research Directive: Research kernel-level proc and sysfs query optimizations on Linux and Android Termux environments to detect CPU throttle states and SIMD extensions without requiring elevated permissions.

ATOM 1.2: ANDROID SHELL ADAPTER AND RUNTIME ENVIRONMENT NEUTRALITY File Path: src/main/kotlin/atropos/core/platform/PlatformAdapter.kt Callers and Interfaces: PlatformAdapter interface, AndroidShellAdapter, AndroidShellAbstraction, JvmStandardAdapter. Values and Symbols: spawnProcess, isAvailable, resolveTool, resolveToolPath, getHomeDirectory, getSystemTemp. Imports and Exports: Imports atropos.core.policy.BoundedProcessRunner, java.nio.file.Path; exports PlatformAbstraction tailored to Android. Implementation Directive: Implement dynamic path resolution for Android bionic libc and Termux environments instead of fixed slash-data paths. Ensure environment variable lookups fall back to application sandboxed storage when system directories are inaccessible. Further Research Directive: Research zero-overhead process spawning using Android NDK posix_spawn shims versus standard ProcessBuilder across Android API level 26 through 35.

ATOM 1.3: DYNAMIC WORKSPACE LOCATION RESOLUTION File Path: src/main/kotlin/atropos/core/TargetWorkspaceLocator.kt Callers and Interfaces: TargetWorkspaceLocator object, invoked by LocalGitMcpTool, AppFactoryRouter, and AgentContextCollector. Values and Symbols: assertAxiomaticIsolation, resolveTargetWorkspace, targetHint Path parameter, engineCapabilities set, targetCapabilities set. Imports and Exports: Imports java.nio.file.Files, java.nio.file.Path, java.util.UUID; exports resolved isolated workspace path. Implementation Directive: Decouple root resolution from hardcoded developer directory paths. Detect fresh runtime states and resolve isolated per-project target directories within user-granted document roots or application-internal cache boundaries. Further Research Directive: Research

Android Storage Access Framework URI tree permission persistence for headless service access without prompting users repeatedly.

ATOM 1.4: FRESH INSTALLATION EMPTY WORKSPACE BOOTSTRAP File Path: apps/atropos-android/src/main/kotlin/atropos/android/bridge/DirectInProcessEngineBridge.kt Callers and Interfaces: DirectInProcessEngineBridge class, called by AndroidSovereignShell and SovereignWorkstationScreen. Values and Symbols: submitGoal, activeProjectName nullable string, isEngineRunning boolean, answers EngineSixAnswers. Imports and Exports: Imports atropos.core.AtroposConfig, atropos.core.agent.AgentService; exports goal dispatch status. Implementation Directive: Ensure fresh initialization launches with a null active project and empty state indicators. Never preload development repositories, mock personal artifacts, or pre-seeded credentials into the runtime state flow. Further Research Directive: Research lightweight state-flow hydration strategies in Jetpack Compose to transition from uninitialized empty workspace to active projects without race conditions.

ATOM 2.1: ENGINE BRIDGE RUNTIME PORT AND ADDRESS RESOLUTION File Path: src/main/kotlin/atropos/bridge/AtroposBridge.kt Callers and Interfaces: LocalEngineBridge object, called by CLI main entrypoint, Web proxy, and Android background service. Values and Symbols: PORT_VARIABLE constant string ATROPOS_BRIDGE_PORT, fromEnvironment, server, boundPort. Imports and Exports: Imports atropos.bridge.http.EngineHttpServer, atropos.core.AtroposConfig; exports running bridge server instance. Implementation Directive: Completely eliminate fallback defaults that bind to developer personal hosts. Read host and port strictly from validated runtime configuration or environment definitions, refusing to bind if invalid or occupied. Further Research Directive: Research local Unix domain socket IPC alternatives to TCP localhost on Android to eliminate loopback port collision risks.

ATOM 2.2: ENGINE REACHABILITY AND INFERENCE DISCRIMINATION File Path: src/main/kotlin/atropos/core/provider/ProviderActivationService.kt Callers and Interfaces: ProviderActivationService class, called by BackendDoctor, StatusCommandHandler, and BridgeRoutes. Values and Symbols: snapshot, verify, verifyAll, liveTest, ProviderActivationState enum, ProviderActivationRecord. Imports and Exports: Imports atropos.core.provider.adapter.ProviderAdapterRegistry, atropos.core.security.RedactionFilter; exports ProviderActivationRecord. Implementation Directive: Enforce strict separation between HTTP transport reachability and model inference execution. A reachable engine bridge must register as bridge connected while downstream neural providers remain explicitly marked untested until verified by actual token exchange. Further Research Directive: Research zero-cost cryptographic heartbeat protocols that ping neural provider endpoints using zero completion tokens to verify authorization without consuming quota.

ATOM 2.3: BRIDGE PROTOCOL ENVELOPE ENCODING AND VALIDATION File Path: src/main/kotlin/atropos/bridge/http/HttpRequestParser.kt

Callers and Interfaces: HttpRequestParser class, invoked by EngineHttpServer accept loops. Values and Symbols: maxRequestLineChars, maxHeaderCount, maxHeaderChars, maxBodyBytes, parse, readHeaders, readBody. Imports and Exports: Imports java.io.BufferedReader, java.net.URLDecoder; exports parsed HttpRequest immutable structures. Implementation Directive: Enforce strict byte ceiling validation on incoming bridge requests. Refuse requests with oversized headers, malicious URL traversal encoding, or unbounded payloads before memory allocation occurs. Further Research Directive: Research streaming zero-copy HTTP header parsing techniques compatible with limited JVM heap profiles on mobile devices.

ATOM 2.4: MOBILE CLIENT BRIDGE SETTINGS CONFIGURATION File Path: app/src/main/java/com/atropos/android/app/ui/SettingsScreen.kt Callers and Interfaces: SettingsScreen composable, invoked by AndroidSovereignShell and navigation drawer. Values and Symbols: bridgeHost text state, bridgePort integer state, isConnected boolean, latencyMs long. Imports and Exports: Imports androidx.compose.runtime, MaterialTheme; exports user-defined bridge network configuration. Implementation Directive: Expose explicit input controls for bridge host and port with immediate validation feedback. Provide connection test utilities that display clear failure diagnostics when connections are refused or timed out. Further Research Directive: Research mDNS and local network discovery protocols (NSD on Android) for discovering local engine instances without manual IP entry.

ATOM 3.1: EMPTY PROJECT CREATION LIFECYCLE File Path: src/main/kotlin/atropos/core/project/ProjectRegistry.kt Callers and Interfaces: ProjectRegistry class, called by ProjectCommandHandler, BridgeRoutes, and AppFactoryRouter. Values and Symbols: register, update, setStatus, ProjectRecord, ProjectStatus enum, currentBinding. Imports and Exports: Imports java.nio.file.Path, atropos.core.security.RedactionFilter; exports updated ProjectRecord. Implementation Directive: Support initializing empty project folders without requiring pre-existing Git remotes or source files. Generate standard project layout descriptors while keeping initial file manifests clean. Further Research Directive: Research transactional directory creation patterns with automatic atomic rollback in case of partial workspace creation failure.

ATOM 3.2: LOCAL STORAGE REPOSITORY IMPORT BOUNDING File Path: src/main/kotlin/atropos/core/ingest/IngestTerritory.kt Callers and Interfaces: IngestTerritory class, called by MentionResolver, FileMentionExpander, and IngestCommand. Values and Symbols: Root data class, Source enum, roots, paths, describe, configuredRoots, pathOrNull. Imports and Exports: Imports java.nio.file.Files, java.nio.file.Path; exports validated IngestTerritory boundaries. Implementation Directive: Restrict project file imports strictly to operator-granted paths such as Android Downloads or designated document directories. Enforce path canonicalization to block directory traversal attacks outside granted roots. Further Research Directive: Research ContentProvider file descriptor streaming for reading external archives into

isolated worktrees without full file duplication.

ATOM 3.3: REMOTE VERSION CONTROL REPOSITORY CLONING File Path: src/main/kotlin/atropos/core/github/GitHubBinding.kt Callers and Interfaces: GitHubBinding class, invoked by WorkerCodeProposalService and SelfHostPromotionService. Values and Symbols: GitHubRepositoryRequest, GitHubPushRequest, GitHubBindingResult, createRepository, push. Imports and Exports: Imports atropos.core.worktree.BoundedGitWorktreeCommandRunner; exports GitHubBindingResult status. Implementation Directive: Decouple remote cloning from hardcoded GitHub assumptions to support arbitrary Git providers including GitLab and self-hosted instances. Isolate credentials during transfer and ensure tokens are not written to git config files on disk. Further Research Directive: Research shallow clone depth optimizations and sparse-checkout configurations to minimize network and disk overhead on mobile devices.

ATOM 3.4: UNIVERSAL IDE PROJECT MANIFEST INTEROPERABILITY File Path: src/main/kotlin/atropos/core/factory/RepoScaffold.kt Callers and Interfaces: RepoScaffold class, called by AppFactoryRouter and AutonomousDeploymentSeam. Values and Symbols: files, ProjectLanguageLayout, forLanguage, layoutFor, spec AppProjectSpec. Imports and Exports: Imports atropos.core.security.RedactionFilter; exports LinkedHashMap of relative file paths and file contents. Implementation Directive: Support reading and generating compatible configuration manifests for VS Code and other editors (.vscode/settings.json, editorconfig). Preserve manifest attributes without stripping custom user properties. Further Research Directive: Research Language Server Protocol (LSP) workspace folder configuration schemas for multi-root project compatibility.

ATOM 3.5: GIT OBJECT AND CACHE FILE FILTERING File Path: src/main/kotlin/atropos/core/security/ContextPathExclusions.kt Callers and Interfaces: ContextPathExclusions object, called by CodebaseContextPacker, AgentContextCollector, and FileExplorer. Values and Symbols: isExcluded, isSourceFile, excludedDirectoryNames set, excludedExtensions set. Imports and Exports: Imports java.lang.String; exports boolean exclusion verdicts for relative paths. Implementation Directive: Distinguish internal Git metadata (.git/objects, .git/index) from legitimate project configuration (.gitignore, .github/workflows). Ensure internal caches are never rendered in the IDE file tree while configuration files remain fully visible and editable. Further Research Directive: Research fast trie-based path pattern matchers that evaluate gitignore specifications concurrently across large file trees.

ATOM 4.1: IDE FILE TREE VISIBILITY AND PROJECTION File Path: apps/web/src/lib/workspace/client.ts Callers and Interfaces: readWorkspaceTree, WorkspaceTreeNode interface, WorkspaceTreePayload, called by FileExplorer component. Values and Symbols: WorkspaceTreeNode data shape, name, path, type, size, children array, readWorkspaceTree. Imports and Exports: Imports readEngine client utility; exports WorkspaceTreePayload to React UI tree

state. Implementation Directive: Render the exact workspace hierarchy without artificially hiding user files based on arbitrary file extension exclusions. Hidden dot-files belonging to user project configs must be listed accurately. Further Research Directive: Research virtualized file tree rendering algorithms capable of smoothly handling directories with over ten thousand entries on low-power mobile viewports.

ATOM 5.1: STRICT ENGINE COMMAND FLOW CONTROL File Path: src/main/kotlin/atropos/bridge/BridgeCommandHandler.kt Callers and Interfaces: BridgeCommandHandler class, called by HttpRouteTable and LocalEngineBridge routing catalogs. Values and Symbols: execute, allowed, run lambda returning BridgeCommandOutput, redactionFilter RedactionFilter. Imports and Exports: Imports atropos.bridge.http.HttpRequest, atropos.core.policy.PortCommandPolicy; exports HttpResponse. Implementation Directive: Ensure all terminal and console commands submitted from Android or Web travel client to engine to client. Never simulate command responses locally on the client. Execute all operations through bounded engine subroutines. Further Research Directive: Research WebSocket binary framing formats to multiplex standard input, standard output, and process control signals over a single connection.

ATOM 5.2: COMMAND PARITY AND CONSISTENT SEMANTICS File Path: src/main/kotlin/atropos/core/parity/SurfaceParityProbe.kt Callers and Interfaces: SurfaceParityProbe class, called by BridgeRoutes and VerifyCommand. Values and Symbols: observe, check, sharedVocabulary, danglingActions, forbiddenOnPort, ParityReport. Imports and Exports: Imports atropos.cli.input.CommandRegistry, atropos.bridge.menu.HelpRegistry; exports ParityReport. Implementation Directive: Validate that commands exposed across Android, CLI, and Web map to the exact same underlying canonical action handlers. Prohibit client-specific divergent command interpretations. Further Research Directive: Research automated AST contract assertion generators that verify command routing tables across Kotlin and TypeScript frontends at build time.

BATCH 2 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (SECTIONS 6 TO 10)

ATOM 6.1: IMMEDIATE SLASH TRIGGER INTERCEPTION File Path: src/main/kotlin/atropos/cli/input/CommandPrefix.kt Callers and Interfaces: CommandPrefix object, called by CommandCompleter, CliModalStateMachine, and InteractiveCliRunner. Values and Symbols: COMMAND_PREFIX_CHARS set, firstNonWhitespace, isCommandPrefixed, InputClassification sealed class. Imports and Exports: Imports standard Kotlin collections; exports classified InputClassification token streams. Implementation Directive: Intercept the forward slash character immediately upon the very first keystroke without requiring any subsequent alphabetic characters or space delimiters. Transition input state instantly to palette navigation mode instead of printing characters to the standard output buffer. Further Research

Directive: Research zero-latency character interception mechanisms in POSIX raw terminal mode across diverse shell environments including Termux, Zsh, Bash, and Windows Terminal.

ATOM 6.2: INTERACTIVE TERMINAL-NATIVE CATEGORY SELECTION UI File Path: src/main/kotlin/atropos/cli/ui/CategorizedTrieModalRenderer.kt Callers and Interfaces: CategorizedTrieModalRenderer class, called by CliModalStateMachine and AnsiTerminalEngine. Values and Symbols: DrillLevel enum, TrieNavState data class, render, renderDomainLevel, renderSubcategoryOrActionLevel. Imports and Exports: Imports atropos.cli.taxonomy.HierarchicalCommandTaxonomy, atropos.cli.ui.design.Role; exports formatted interactive string lines. Implementation Directive: Construct a terminal-native modal viewport overlay displaying top-level command categories with persistent navigation chrome. Render keyboard shortcuts visibly at all times across all drill levels. Further Research Directive: Research double-buffered ANSI escape sequence rendering to eliminate visual screen flicker during modal state transitions on mobile terminals.

ATOM 6.3: MULTI-LEVEL TRIE NAVIGATION AND ARROW-KEY HIERARCHY File Path: src/main/kotlin/atropos/cli/taxonomy/HierarchicalCommandTaxonomy.kt Callers and Interfaces: HierarchicalCommandTaxonomy class, invoked by CommandCompleter, CategorizedTrieModalRenderer, and HelpRegistry. Values and Symbols: TaxonomyNode data class, root map, register, stepInto, resolveLeaf, cheatsheet. Imports and Exports: Imports atropos.cli.input.CommandEntry; exports hierarchical TaxonomyNode trees and leaf descriptions. Implementation Directive: Build a multi-tier tree structure mapping domain to action to target. Ensure right-arrow and enter keys descend into child categories, left-arrow ascends to parent nodes, and escape dismisses the hierarchy cleanly. Further Research Directive: Research compact prefix trie data compression structures for instant symbol lookups across several thousand command definitions with sub-millisecond latency.

ATOM 6.4: DETAILED COMMAND INSPECTOR AND PARITY CARD DISPLAY File Path: src/main/kotlin/atropos/cli/help/CommandLeafDocumentation.kt Callers and Interfaces: CommandLeafRegistry object, invoked by CategorizedTrieModalRenderer and CommandHelpRenderer. Values and Symbols: CommandLeafDocumentation data class, command, brief, syntax, examples list, sideEffects, securityRisk, costClass. Imports and Exports: Imports atropos.cli.ui.TerminalText, atropos.cli.ui.TerminalTheme; exports formatted multi-line command specification cards. Implementation Directive: Present comprehensive command metadata upon right-arrow navigation on a leaf node. Display full syntax usage, execution risks, required security capabilities, affected territories, and practical examples matching the Android app details. Further Research Directive: Research terminal Markdown layout wrapping algorithms that preserve code block margins and table alignment dynamically when rendered on narrow screens.

ATOM 6.5: BUFFER INSERTION WITHOUT AUTO-EXECUTION File

Path: src/main/kotlin/atropos/cli/input/PromptTextBuffer.kt Callers and Interfaces: PromptTextBuffer class, called by PromptState and InteractiveCliRunner. Values and Symbols: insert, textBeforeCursor, moveEnd, replace, cursor, text, maximumLength. Imports and Exports: Imports standard Kotlin string primitives; exports raw buffer state to CLI rendering engine. Implementation Directive: When an operator confirms selection of a leaf command from the palette, inject the full command text into the prompt buffer and position the cursor at the end. Never execute the command automatically upon selection, allowing the user to review, edit, or append flags. Further Research Directive: Research IME composition string coordination in terminal emulators to avoid breaking text cursor positions during programmatic buffer insertions.

ATOM 6.6: CATALOG COMMAND SYNCHRONIZATION AND ALIGNMENT File Path: src/main/kotlin/atropos/cli/input/CommandCatalog.kt Callers and Interfaces: CommandCatalog object, read by CommandRegistry and BridgeRouteCatalog. Values and Symbols: catalog list of CommandEntry objects, command, description, category, aliases, risk. Imports and Exports: Imports atropos.cli.input.CommandEntry, atropos.cli.input.CommandRisk; exports immutable catalog collection. Implementation Directive: Synchronize command definitions across engine, Android, and web frontends. Register valid engine commands including flash and self-host subcommands while pruning unsupported placeholder aliases. Further Research Directive: Research compile-time code generation of command descriptors from Kotlin Multiplatform definitions to guarantee strict syntax parity across native, JVM, and TypeScript targets.

ATOM 7.1: DYNAMIC ENVIRONMENTAL HARDWARE PROFILING File Path: src/main/kotlin/atropos/core/platform/PlatformModels.kt Callers and Interfaces: PlatformDescriptor data class, detect method, PlatformCapability enum, RuntimePlatform enum. Values and Symbols: detect, defaultCapabilities, platform, cpuCores, totalMemoryBytes, isLowRamDevice. Imports and Exports: Imports java.nio.file.Path; exports immutable PlatformDescriptor telemetry. Implementation Directive: Query live system properties and runtime interfaces at boot to determine actual hardware parameters. Eliminate fixed desktop defaults and adapt system memory limits based on whether the host is a mobile container or physical server. Further Research Directive: Research lightweight Linux cgroup and Android thermal zone monitoring hooks to detect thermal throttling events dynamically during heavy background task runs.

ATOM 7.2: TIMEZONE-AWARE LOCAL TIME GREETINGS ENGINE File Path: src/main/kotlin/atropos/cli/ui/StartupSequence.kt Callers and Interfaces: StartupSequence class, called by AnsiStartupPlayer and AnsiTerminalEngine. Values and Symbols: clock supplier returning LocalTime, greetingLine, factLines, frames, finalFrame. Imports and Exports: Imports java.time.LocalTime, java.time.ZoneId; exports animated startup frame

sequences. Implementation Directive: Inspect the device local clock and timezone dynamically at launch. Select contextual greetings such as Good morning, Good afternoon, or context-aware evening notices without assuming developer timezone offsets. Further Research Directive: Research low-power clock synchronization techniques across headless devices lacking battery-backed real-time clock hardware.

ATOM 7.3: ADAPTIVE INTERFACE INPUT MODE DETECTION File Path: src/main/kotlin/atropos/core/output/OutputModeDetector.kt Callers and Interfaces: OutputModeDetector object, called by AnsiTerminalEngine and Phase20Authorization. Values and Symbols: OutputMode enum, detect, interactive boolean, supportsAnsi boolean, isHeadless boolean. Imports and Exports: Imports java.lang.System; exports resolved OutputMode configuration. Implementation Directive: Distinguish interactive TTY sessions from headless CI environments, piped streams, or touch-driven terminal wrappers. Adjust interactive prompts, redraw behaviors, and escape sequences accordingly. Further Research Directive: Research Terminfo database fallback mechanisms for non-standard mobile terminal emulators that fail standard VT100 capability queries.

ATOM 8.1: INSTANT MOBILE PASTE INGESTION File Path: src/main/kotlin/atropos/cli/input/FuzzyMat
Callers and Interfaces: FuzzyMatcher class, invoked by InteractiveCliRunner and ByteZeroPrefixInterceptor. Values and Symbols: BRACKETED_PASTE_ENABLE, BRACKETED_PASTE_DISABLE, processPaste, Utf8RingBufferDecoder. Imports and Exports: Imports java.nio.charset.StandardCharsets; exports processed paste text and optional BlobObject. Implementation Directive: Ingest pasted text streams immediately upon arrival via bracketed paste markers. Eliminate all requirements for secondary keyboard events like arrow down presses to trigger buffer rendering. Further Research Directive: Research terminal escape sequence buffering behaviors under high-throughput paste operations to prevent buffer overrun and dropped newline bytes.

ATOM 8.2: BRACKETED PASTE DEMUXING AND LARGE PASTE MANAGEMENT File Path: src/main/kotlin/atropos/cli/input/CollapsedPasteRegistry.kt
Callers and Interfaces: CollapsedPasteRegistry class, called by PromptState and PromptHistoryBrowser. Values and Symbols: lineThreshold, wordThreshold, shouldCollapse, collapse, expand, hasCollapsedText. Imports and Exports: Imports java.util.LinkedHashMap; exports collapsed text tokens and expansion lookups. Implementation Directive: Detect large multiline paste events and replace them with interactive single-line token placeholders to preserve viewport space. Allow the operator to expand, edit, or submit the pasted content seamlessly. Further Research Directive: Research memory-mapped temporary storage backing for multi-megabyte clipboard insertions to prevent JVM garbage collection spikes on mobile hardware.

ATOM 8.3: TOUCH-SAFE MOBILE KEYBOARD FOCUS AND IME HANDLING File Path: app/src/main/java/com/atropos/android/app/ui/MobileComposer.kt

Callers and Interfaces: ComposerScreen composable, invoked by SovereignWorkstationScreen and ConversationScreen. Values and Symbols: ComposerScreen, canSendComposerInput, OneHandDensity, touchTargetDp. Imports and Exports: Imports androidx.compose.material3, androidx.compose.ui.Modifier; exports mobile composer layout. Implementation Directive: Configure software keyboard options with appropriate capitalization and input actions. Maintain persistent input focus without accidental blur events when touch gestures occur nearby. Further Research Directive: Research Android WindowInsetsCompat animation listener integration to smoothly resize Compose message lists during keyboard appearance without layout stutter.

ATOM 9.1: INPUT-ISOLATED OUTPUT GESTURE DECOUPLING File Path: src/main/kotlin/atropos/cli/ui/TerminalCanvas.kt Callers and Interfaces: TerminalCanvas class, invoked by AnsiTerminalEngine and TerminalRenderingFacade. Values and Symbols: disableMouseTracking, enableMouseTracking, render, refreshGeometry, width, height. Imports and Exports: Imports java.io.PrintStream, atropos.cli.config.ConfigurationManager; exports canvas coordinate frames. Implementation Directive: Decouple mouse tracking and touch-swipe escape sequences originating in the scrollback area from the active prompt input buffer. Prevent scroll gestures from emitting accidental control characters into the composer. Further Research Directive: Research SGR mouse mode protocol handling in terminal emulators to cleanly differentiate drag gestures from click and cursor repositioning events.

ATOM 9.2: BOUNDED INERTIAL SCROLLING AND SPIN TERMINATION File Path: src/main/kotlin/atropos/cli/ui/TranscriptBuffer.kt Callers and Interfaces: TranscriptBuffer class, called by TranscriptRenderer, AnsiTerminalEngine, and ViewportLayout. Values and Symbols: scrollUp, scrollDown, followTail, currentScrollOffset, isFollowingTail, visibleLines. Imports and Exports: Imports atropos.cli.ui.TranscriptEntry; exports scrollback line slices. Implementation Directive: Clamp scroll offsets strictly between zero and the maximum buffered output lines. Terminate touch fling inertia cleanly when reaching boundary edges, avoiding infinite scroll loops or unresponsive touch states. Further Research Directive: Research decayed velocity physics models for touch scroll fling gestures to ensure natural deceleration curves in software-rendered terminal viewports.

ATOM 9.3: GESTURE-SAFE VIEWPORT LAYOUT AND FOCUS LOCK File Path: src/main/kotlin/atropos/cli/ui/ViewportLayout.kt Callers and Interfaces: ViewportLayout class, called by AnsiTerminalEngine and ReactiveScreenComposer. Values and Symbols: build, safeWidth, safeHeight, contentStart, transcriptStart, transcriptHeight, composerStart. Imports and Exports: Imports atropos.cli.ui.chrome.StickyRegionSolver, atropos.cli.ui.TerminalTheme; exports ScreenFrame. Implementation Directive: Enforce rigid separation between pinned header chrome, transcript viewport, and input composer. Ensure scrolling gestures within the transcript area never alter the layout boundaries or push the composer off-screen. Further Research

Directive: Research sub-window clipping optimizations in ANSI terminal drivers to repaint only modified scroll lines rather than re-rendering the full screen buffer.

ATOM 10.1: BIDIRECTIONAL FEATURE AND COMMAND ALIGNMENT File Path: src/main/kotlin/atropos/core/parity/CrossSurfaceParity.kt Callers and Interfaces: CrossSurfaceParity class, invoked by SurfaceParityProbe and CI contract verification test suites. Values and Symbols: StatusVocabulary enum, ProjectIdentity data class, validateStatus, getStatusDisplay, validateProjectIdentity. Imports and Exports: Imports standard Kotlin collections; exports cross-surface parity validation reports. Implementation Directive: Ensure all functional operations present in the Android app are accessible via CLI slash commands and engine RPC routes, and conversely ensure CLI tools are exposed across GUI surfaces. Further Research Directive: Research declarative contract schema languages to define shared cross-platform surface capabilities with automated parity test generation.

ATOM 10.2: UNIFIED STATUS AND STATE PRESENTATION MAPPING File Path: src/main/kotlin/atropos/cli/ui/design/HoeStatusVocabulary.kt Callers and Interfaces: HoeStatusVocabulary object, called by StatusBadge, SixAnswersProjection, and DashboardRenderer. Values and Symbols: CANONICAL_TERMS list, resolve, signal, legend, conformance, StatusSignal data class. Imports and Exports: Imports atropos.cli.ui.design.RunState; exports normalized status symbols and human-readable text. Implementation Directive: Map engine states to unified canonical terms across mobile, web, and CLI interfaces. Disallow client-specific state renaming that obscures true operational status. Further Research Directive: Research WCAG 2.1 non-color state indicator compliance testing to ensure status signals remain completely distinguishable under full color blindness conditions.

ATOM 10.3: ENGINE CAPABILITY PARITY VERIFICATION File Path: src/test/kotlin/atropos/core/parity/SurfaceContractParityTest.kt Callers and Interfaces: SurfaceContractParityTest test class, executed during continuous integration test runs. Values and Symbols: canonical10SpineIds list, verifySpineParity, checkSurfaceContract, assertVocabularyMatches. Imports and Exports: Imports atropos.core.parity.SurfaceContract, org.junit.Test; exports test assertion receipts. Implementation Directive: Implement automated test suites that verify primary navigation spine identity, command routing tables, and completion claims between Android Compose and CLI surfaces. Fail builds on semantic drift. Further Research Directive: Research automated AST symbol diffing tools that compare Kotlin Compose screen hierarchies against CLI ANSI view tree models to detect unmapped surface components.

BATCH 3 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (SECTIONS 11 TO 15)

ATOM 11.1: SPECGRAPH FOUNDRY NESTED PROJECT ROOT CONFIGURATION File Path: apps/specgraph-foundry/apps/web/next.config.ts

Callers and Interfaces: Vercel build engine, NextConfig schema, invoked during CI and remote deployments. Values and Symbols: nextConfig, securityHeaders, poweredByHeader, reactStrictMode, typedRoutes, productionBrowserSourceMaps. Imports and Exports: Imports NextConfig type from next; exports default nextConfig configuration object. Implementation Directive: Configure the nested Next.js build root so that Vercel resolves dependencies, assets, and route manifests strictly within apps/specgraph-foundry/apps/web. Ensure build commands and asset output directories do not search the parent repository root or overwrite the Atropos web workspace. Further Research Directive: Research Vercel Monorepo Project Root settings and Turborepo workspace filter caching to optimize isolated sub-package deployments without pulling unnecessary monorepo dependencies.

ATOM 11.2: VERCEL PRODUCTION ENVIRONMENT AND PUBLIC VARIABLE ISOLATION File Path: apps/specgraph-foundry/apps/web/src/lib/config/client-env.ts Callers and Interfaces: readPublicEnv function, called by createSupabaseBrowserClient, SpecGraphApiClient, and AppProviders. Values and Symbols: publicEnvSchema, NEXT_PUBLIC_SPECGRAPH_API_URL, NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY. Imports and Exports: Imports z from zod; exports readPublicEnv function and PublicEnv type definition. Implementation Directive: Validate that all public environment variables strictly point to production domains over HTTPS. Prohibit fallback defaults to localhost or developer machines in production mode. Throw informative validation errors if required Vercel project secrets are missing. Further Research Directive: Research automated runtime environment schema validation in edge middleware to catch configuration drift before requests hit client rendering logic.

ATOM 11.3: VERCEL PRODUCTION BUILD GUARD AND MANIFEST VALIDATION File Path: apps/web/src/lib/architecture/vercel-deploy-guard.ts Callers and Interfaces: assertVercelBuildSafe, verifySpecGraphPipelineRoutes, called during pre-build scripts and health probes. Values and Symbols: VercelBuildEnvironmentCheck, SpecGraphPipelineIntegrity, sourcesAccessible, researchAccessible, graphWorkspaceAccessible, handoffAccessible, routingAccessible. Imports and Exports: Exports deployment verification functions and environment diagnostic interfaces. Implementation Directive: Implement explicit integrity assertions that run prior to deployment bundling. Verify that all five core pipeline route segments exist and resolve cleanly, preventing partial builds from deploying to production. Further Research Directive: Research static route manifest analysis tools within Next.js AST trees to verify deep link integrity at compile time.

ATOM 11.4: DEPLOYMENT VERIFICATION AND ZERO-ENTROPY OUTPUT VALIDATION File Path: apps/specgraph-foundry/apps/web/src/lib/execution/mutations.ts Callers and Interfaces: useVerifyExecutionRunMutation hook, called by ExecutionRunDetail and HandoffWorkspace. Values and Symbols: verifyExecutionRun, useVerifyExecutionRunMutation, projectId, runId, onProgress callback.

Imports and Exports: Imports useMutation, useQueryClient from tanstack react-query; exports mutation hook. Implementation Directive: Verify that successful production deployments produce verifiable cryptographic evidence. Ensure the deployment receipt records the commit SHA, manifest hash, and asset digest into the tamper-evident log rather than assuming success from an HTTP 200 response alone. Further Research Directive: Research Merkle tree leaf verification of static asset bundles delivered via CDN edges to guarantee content-addressed release attestation.

ATOM 12.1: ARCHITECTURAL SEPARATION AND FORBIDDEN RUNTIME ROOT ENFORCEMENT File Path: apps/web/src/lib/architecture/webmerge.ts Callers and Interfaces: WebMergeArchitecture object, invoked by navigation routing guards and architecture unit tests. Values and Symbols: canonicalRoot, forbiddenRuntimeRoots array, developerToolsPrefix, isDeveloperToolsRoute, isCanonicalRuntimeRoot. Imports and Exports: Exports immutable architectural constants and runtime path validation methods. Implementation Directive: Maintain strict boundary enforcement preventing the SpecGraph Foundry standalone website from colliding with the local Atropos IDE web server. Route SpecGraph tools inside the Atropos cockpit exclusively under the developer tools namespace while leaving the standalone Foundry site independently deployable. Further Research Directive: Research Next.js multi-zone deployment architectures where separate web applications share a unified parent domain without code-level runtime coupling.

ATOM 12.2: SPECGRAPH DEVELOPER TOOLS TENANCY NAMESPACE File Path: apps/web/src/lib/specgraph/tenancy.ts Callers and Interfaces: SPECGRAPH_TENANCY_PREFIX constant, tenancyFindings, tenantedWorkspaces, called by AppSidebar and DevToolsContainer. Values and Symbols: SPECGRAPH_TENANCY_PREFIX string, TenancyFinding interface, CARRIED_OVER list, isSpecGraphRoute. Imports and Exports: Imports globalRoutes, navigationSpine; exports tenancy resolution utilities. Implementation Directive: Encapsulate all embedded SpecGraph workspaces beneath the developer tools hierarchy. Ensure navigation badges, workspace state, and project selectors keep SpecGraph subordinate to the core Atropos development environment. Further Research Directive: Research dynamic micro-frontend iframe sandboxing versus module federation for isolating sub-tools with conflicting CSS utility versions.

ATOM 12.3: PRODUCTION FOUNDRY URL INTEGRATION ENDPOINT File Path: apps/web/src/components/dev-tools/specgraph-entry.tsx Callers and Interfaces: SpecGraphEntry component, rendered within DeveloperToolsContainer and ProjectOverview. Values and Symbols: projectGraphRoute, projectHandoffRoute, projectSourcesRoute, WebMergeArchitecture. Imports and Exports: Imports Link from next link, useSessionState hook; exports SpecGraphEntry visual entrypoint. Implementation Directive: Provide a clean launch point connecting users to the live deployed SpecGraph Foundry platform. Ensure the outbound connection uses the verified production URL

resolved from configuration and clearly distinguishes the external website from local IDE operations. Further Research Directive: Research secure cross-origin message passing tokens to allow seamless single-sign-on handoff between the local IDE and the cloud Foundry platform.

ATOM 13.1: IDE FOUR-PANE WORKBENCH SHELL ARCHITECTURE File Path: apps/web/src/components/workbench/workbench-shell.tsx Callers and Interfaces: WorkbenchShell component, invoked by WorkbenchMain and AppShell layouts. Values and Symbols: WorkbenchSlots interface, explorer slot, editor slot, logs slot, aiRail slot, terminal slot. Imports and Exports: Imports ReactNode from react; exports WorkbenchShell container component. Implementation Directive: Build a dense, highly responsive four-pane IDE layout mirroring modern desktop development environments. Support collapsible panels, flexible splitter dividers, and fluid CSS grid resizing to maximize workspace productivity. Further Research Directive: Research CSS subgrid and container query optimization strategies to prevent layout reflow thrashing when resizing multi-pane code editors.

ATOM 13.2: HIERARCHICAL FILE EXPLORER WITH IN-TERRITORY CONTEXT File Path: apps/web/src/components/workbench/file-explorer.tsx Callers and Interfaces: FileExplorer component, called by WorkbenchShell and SovereignIdeWorkstation. Values and Symbols: WorkspaceTreeNode interface, TreeList, DirectoryNode, FileNode, readWorkspaceTree, readWorkspaceFile. Imports and Exports: Imports useWorkbenchTabs, readEngine, writeWorkspaceFile; exports FileExplorer tree view. Implementation Directive: Display an interactive, virtualized file tree representing the active workspace. Clearly mark in-territory writable files versus out-of-territory read-only files, allowing users to browse, create, rename, and delete project files with live status indicators. Further Research Directive: Research file-system event streaming over WebSockets using fsnotify primitives to update client-side tree nodes instantly without periodic polling.

ATOM 13.3: MULTI-TAB DIRTY STATE AND SPLIT CODE EDITOR File Path: apps/web/src/components/workbench/editor-tabs.tsx Callers and Interfaces: EditorTabs component, rendered in WorkbenchShell editor slot. Values and Symbols: useWorkbenchTabs, writeWorkspaceFile, openTab, closeTab, markClean, setContent. Imports and Exports: Imports useWorkbenchTabs context hook; exports EditorTabs control panel and active buffer view. Implementation Directive: Support multiple simultaneous editor tabs with dirty state indicators, syntax highlighting, line numbering, and split horizontal or vertical views. Synchronize tab modifications directly to disk via atomic write operations. Further Research Directive: Research Monaco Editor Web Worker optimization techniques to offload AST parsing and linter calculations from the browser main UI thread.

ATOM 13.4: REAL ENGINE MIRROR TERMINAL AND LOG STREAM PANEL File Path: apps/web/src/components/workbench/terminal.tsx Callers and Interfaces: TerminalComponent, useTerminal hook, rendered in Work-

benchShell bottom panel. Values and Symbols: Terminal from xterm, FitAddon, WebLinksAddon, SerializeAddon, spawnUnixPty. Imports and Exports: Imports xterm and xterm addons; exports TerminalComponent and useTerminal controller. Implementation Directive: Connect the integrated web terminal directly to the active engine process via WebSocket stream. Reflect actual shell execution, stdout, stderr, and exit codes without simulating local terminal responses. Further Research Directive: Research xterm.js WebGL addon integration for hardware-accelerated 60 FPS terminal rendering during massive compiler diagnostic output dumps.

ATOM 14.1: ANDROID 3-TO-5 SECOND ANIMATED STARTUP SEQUENCE File Path: app/src/main/java/com/atropos/android/app/theme/AtroposCyberTheme.kt Callers and Interfaces: AtroposCyberTheme composable, invoked by MainActivity on cold start. Values and Symbols: CyberHudColors data class, neonPulseBorder, cyberGridBackground, alpha animation, transition. Imports and Exports: Imports androidx.compose.animation.core, androidx.compose.ui.graphics; exports themed container modifiers. Implementation Directive: Implement an engaging, cinematic startup sequence lasting three to five seconds on cold launch. Render coordinated glowing cyber-grid animations and branded visual motifs to establish product identity before transitioning to input states. Further Research Directive: Research Android Compose graphics layer hardware acceleration shaders (RenderEffect) to achieve cinematic glow effects with zero CPU overhead.

ATOM 14.2: EXPLICIT ENTER BUTTON GATE AND APPLICATION TRANSITION File Path: apps/atropos-android/src/main/kotlin/atropos/android/ui/AndroidSovereignShell.k Callers and Interfaces: AndroidSovereignShell composable, called after startup animation completion. Values and Symbols: AndroidShellViewState data class, QuickCommandStrip, ContinuousAnswersGrid, isEngineRunning. Imports and Exports: Imports androidx.compose.foundation, androidx.compose.material3; exports shell screen composable. Implementation Directive: Conclude the startup sequence with a distinct, accessible Enter button. Require explicit user touch confirmation to enter the workspace, transitioning fluidly from the introductory sequence into the main workstation. Further Research Directive: Research Compose SharedTransitionLayout APIs to animate the Enter button organically into the primary command bar during the workspace entrance transition.

ATOM 15.1: PUBLIC LANDING PAGE AND MARKETING FUNNEL ARCHITECTURE File Path: apps/web/src/components/landing/landing-hero.tsx Callers and Interfaces: LandingHero component, rendered on unauthenticated root home page routes. Values and Symbols: FEATURES array, UploadIcon, EvidenceIcon, GraphIcon, VerifiedIcon, LandingHero. Imports and Exports: Imports Link from next link, Button, Card; exports LandingHero layout component. Implementation Directive: Deliver an informative, conversion-oriented public landing page explaining the capabilities of Atropos and SpecGraph Foundry. Present clear value propositions, architec-

tural diagrams, and documentation links without demanding authentication for public viewing. Further Research Directive: Research Next.js Static Site Generation (SSG) with edge caching for public marketing landing pages to achieve sub-50ms global time-to-first-byte (TTFB).

ATOM 15.2: PUBLIC ROUTE PROXY AND AUTHENTICATION WALL File Path: apps/web/src/lib/auth/proxy.ts Callers and Interfaces: proxy middleware, safeRedirectPath, refreshSession, isPublicPath. Values and Symbols: safeRedirectPath, refreshSession, isPublicPath, publicPaths set, sessionCookie. Imports and Exports: Imports NextResponse, NextRequest, createServerClient; exports route protection middleware. Implementation Directive: Enforce an airtight boundary between public marketing pages and protected application routes. Allow unrestricted access to the landing page, documentation, and auth flows while strictly redirecting unauthenticated attempts on protected workspaces to sign-in. Further Research Directive: Research zero-latency session verification in Vercel Edge Middleware using signed JWT claims without executing database roundtrips.

ATOM 15.3: AUTHENTICATED APPLICATION SHELL TRANSITION File Path: apps/web/src/app/(app)/layout.tsx Callers and Interfaces: AppLayout component, wraps all protected application workspaces and navigation bars. Values and Symbols: AppLayoutProps, getVerifiedUser, redirect, AppShell, SessionTabBar. Imports and Exports: Imports getVerifiedUser from auth server, redirect from next navigation; exports AppLayout component. Implementation Directive: Verify active user session credentials prior to mounting the primary IDE shell. Upon successful verification, hydrate the user profile, active project bindings, and workspace layout theme into the application context. Further Research Directive: Research optimistic UI hydration patterns in React Server Components to eliminate layout shift when transitioning from public auth screens into the authenticated workbench.

BATCH 4 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (SECTIONS 16 TO 20)

ATOM 16.1: DEMONSTRATION WORKSPACE SCAFFOLDING AND VISUAL ONBOARDING File Path: apps/web/src/components/welcome/welcome-panel.tsx Callers and Interfaces: WelcomePanel component, invoked by AppShell and Home entrypoints. Values and Symbols: hasSeen, markSeen, welcomeClient, WelcomePayload, SEEN_STORAGE_KEY. Imports and Exports: Imports welcome client from lib/welcome/client; exports WelcomePanel React component. Implementation Directive: Render clear demonstration workspace artifacts on fresh installs before user projects exist. Display illustrative file trees, representative README guidelines, and feature previews labeled explicitly as sample demonstration data. Further Research Directive: Research interactive onboarding walkthrough overlays with localized spotlight cues that guide first-time developers through key workspace panels.

ATOM 16.2: STRICT SEGREGATION OF MOCK AND PRODUCTION

WORKSPACES File Path: apps/web/src/components/projects/project-empty-state.tsx Callers and Interfaces: ProjectEmptyState component, rendered by ProjectDirectory and ProjectCommandCenter. Values and Symbols: ProjectEmptyState, projectRoute, Button, Card, emptyStateText. Imports and Exports: Imports Link from next link, UI primitives; exports ProjectEmptyState component. Implementation Directive: Maintain strict isolation between demonstration data and real repository state. Sample files must never trigger build pipelines, commit hooks, or automated engine tasks, and must vanish completely once a real project is imported. Further Research Directive: Research client-side ephemeral in-memory filesystem overlays for demo sandboxing that guarantee zero residual state on physical storage.

ATOM 16.3: MOBILE FRESH-INSTALL ORIENTATION AND GUIDANCE HINTS File Path: app/src/main/java/com/atropos/android/app/ui/ConversationScreen.kt Callers and Interfaces: EmptyConversationHint composable, called by ConversationScreen when message transcripts are empty. Values and Symbols: EmptyConversationHint, conversationStatus, isOnline boolean, input state. Imports and Exports: Imports androidx.compose.material3, androidx.compose.runtime; exports contextual empty state hints. Implementation Directive: Provide explanatory onboarding guidance on mobile viewports when transcripts or work lists are empty. Guide the user to import a project or configure a bridge endpoint without presenting confusing blank screens. Further Research Directive: Research predictive onboarding suggestions based on detected device capabilities (such as local storage permissions or network reachability).

ATOM 17.1: UNIFIED SUPABASE AUTHENTICATION AND SESSION PERSISTENCE File Path: apps/web/src/lib/auth/server.ts Callers and Interfaces: createSupabaseServerClient, getVerifiedUser, called by ApplicationLayout and API routes. Values and Symbols: cookies from next headers, createServerClient, readServerEnv, user profile claims. Imports and Exports: Imports Supabase SSR client primitives; exports server-side session authentication helpers. Implementation Directive: Connect authentication flows to the central Supabase PostgreSQL account database. Support persistent user sessions across Android, Web, and CLI interfaces while ensuring sensitive session cookies remain HTTP-only and securely signed. Further Research Directive: Research PKCE OAuth flows on mobile clients to enable passwordless biometrics and hardware security key authentication via Supabase Auth.

ATOM 17.2: CROSS-PLATFORM CREDENTIAL SYNC AND EDGE TOKEN ROTATION File Path: apps/web/src/lib/auth/session.ts Callers and Interfaces: markSessionExpired, resetSessionExpiredFlag, isSessionExpiredStatus, called by QueryProvider and API fetchers. Values and Symbols: sessionExpiredShown boolean, markSessionExpired, resetSessionExpiredFlag, isSessionExpiredStatus. Imports and Exports: Imports QueryClient from tanstack react-query; exports session expiration lifecycle controllers. Implementation Directive: Manage token refresh cycles seamlessly across all active surfaces. Intercept expired token responses at the network boundary,

refresh claims transparently, and surface explicit re-authentication dialogs only when sessions become permanently invalid. Further Research Directive: Research silent background token refresh patterns in Web Workers and Android JobScheduler to prevent user interruption during active coding sessions.

ATOM 17.3: ROLE-BASED AUTHORIZATION AND OWNER IDENTITY SEPARATION File Path: src/main/kotlin/atropos/core/security/OwnerAuthorizationService.kt Callers and Interfaces: OwnerAuthorizationService class, called by ProtectedPathPolicy, SelfHostCommand, and Phase20Authorization. Values and Symbols: isConfigured, setup, verifyPassword, requireAuthorization, validateToken, AuthorizationToken. Imports and Exports: Imports java.security.SecureRandom, javax.crypto.spec.PBEKeySpec; exports AuthorizationToken models. Implementation Directive: Distinguish ordinary authenticated developer accounts from project owner administrative roles. Enforce cryptographic proof-of-authority before allowing access to administrative subroutines, ensuring regular users cannot invoke owner-only capabilities. Further Research Directive: Research multi-factor authorization mechanisms using time-based one-time password (TOTP) algorithms integrated with sovereign local vault stores.

ATOM 18.1: COMPREHENSIVE MULTI-PROVIDER SETTINGS MATRIX File Path: apps/web/src/app/(app)/settings/page.tsx Callers and Interfaces: SettingsMatrixPage component, rendered under the global settings navigation spine. Values and Symbols: ProviderConfig interface, providers list, envKey, tier, status, latencyMs, testConnection. Imports and Exports: Imports Lucide icons, session state hooks; exports SettingsMatrixPage view. Implementation Directive: Present a comprehensive administrative interface exposing all supported model providers, local engines, and remote cloud gateways. Display accurate connectivity states, current latency readings, and token burn quotas for each configured integration. Further Research Directive: Research parallel asynchronous health checking architectures that query multiple provider status endpoints without blocking UI interactivity.

ATOM 18.2: FIRST-FOUR LAST-FOUR KEY MASKING AND REDACTION GATE File Path: src/main/kotlin/atropos/core/security/RedactionFilter.kt Callers and Interfaces: RedactionFilter class, called by ProviderActivationRecord, SettingsMatrixPage, and ErrorRenderer. Values and Symbols: redact, compact, report, RedactionReport, RedactionFinding, stableFingerprint. Imports and Exports: Imports java.security.MessageDigest; exports redacted strings and audit findings. Implementation Directive: Enforce strict API key masking across all visual surfaces. When an operator triggers an eye reveal toggle, reveal at most the first four and last four characters with bullet masking between. Never expose complete secrets in HTML attributes, JSON responses, or memory logs. Further Research Directive: Research client-side memory scrubbing techniques in JavaScript and Kotlin to zero out sensitive credential strings immediately after cryptographic hashing.

ATOM 18.3: ISOLATED VAULT STORAGE AND KEY SANITIZATION File

Path: src/main/kotlin/atropos/core/security/TokenIsolationVault.kt Callers and Interfaces: TokenIsolationVault class, called by CredentialVault, McpHostManager, and ProviderOnboardingService. Values and Symbols: readSecret, writeSecret, inspectSecret, deleteSecret, vaultHealth, TokenIsolationReport. Imports and Exports: Imports java.nio.file.Files, java.nio.file.Path; exports secure vault read and write outcomes. Implementation Directive: Isolate stored API keys and service tokens on physical disk using AES-256-GCM encryption with platform-restricted POSIX file permissions. Reject symlink traversal attempts and prevent unauthorized sub-processes from inspecting secret payloads. Further Research Directive: Research hardware-backed Android KeyStore and Linux kernel keyring APIs to store vault master keys in tamper-resistant enclaves.

ATOM 19.1: DEDICATED SPECGRAPH FOUNDRY PRODUCTION PORTAL LINKAGE File Path: apps/web/src/components/dev-tools/specgraph-entry.tsx Callers and Interfaces: SpecGraphEntry component, rendered within the developer tools drawer. Values and Symbols: SpecGraphEntry, projectGraphRoute, projectHandoffRoute, WebMergeArchitecture. Imports and Exports: Imports Next navigation routes; exports interactive SpecGraph portal launcher. Implementation Directive: Provide a dedicated, prominent interface entrypoint connecting operators to the verified SpecGraph Foundry cloud deployment. Ensure the outbound link points strictly to the production Vercel URL and clearly explains the relationship between Atropos IDE and Foundry planning. Further Research Directive: Research web deep-linking protocols and custom URL schemes to launch specific graph projects in the Foundry directly from local IDE context.

ATOM 19.2: PRODUCTION ENDPOINT RESOLUTION AND ENVIRONMENT BOUNDING File Path: apps/web/src/lib/specgraph/client.ts Callers and Interfaces: specgraph client object, invoked by SpecGraphProjectView and DeveloperToolsContainer. Values and Symbols: SpecGraphProject, SpecGraphSource, SpecGraphAtom, SpecGraphExecution, SpecGraphPlan. Imports and Exports: Imports readEngine client; exports typed SpecGraph API query handlers. Implementation Directive: Resolve SpecGraph Foundry endpoints strictly against declared production configurations. Reject stale development server URLs or unverified localhost ports, ensuring cloud synchronization requests reach authentic Foundry infrastructure. Further Research Directive: Research mutual TLS (mTLS) certificate exchange protocols to authenticate direct engine-to-cloud synchronization channels against unauthorized tampering.

ATOM 20.1: AUTONOMOUS BUG DISCOVERY AND DIAGNOSTIC SLICING ENGINE File Path: src/main/kotlin/atropos/core/verification/DiagnosticSymbolSlicer.kt Callers and Interfaces: DiagnosticSymbolSlicer object, called by CompilerDiagnosticRepairLoop and SentryRepairCoordinator. Values and Symbols: slice, SlicedDiagnostic data class, parseDiagnosticTrace, extractCodeSlice, buildCompressedPrompt. Imports and Exports: Imports java.nio.file.Path, java.util.regex.Pattern; exports SlicedDiagnostic context blocks. Implemen-

tation Directive: Parse raw compiler diagnostics and error stack traces into structured syntax slices. Extract the precise failing code scope (functions, classes, or blocks within a 25-line window) to construct minimal, focused prompts for autonomous repair models. Further Research Directive: Research tree-sitter based semantic scope analysis to isolate dependent variable declarations across multiple files for complex cross-module bug diagnosis.

ATOM 20.2: NATIVE MULTI-LANGUAGE TOOLCHAIN AND SDK DRIVER REGISTRY File Path: src/main/kotlin/atropos/core/toolchain/NativeToolchainRegistry.kt Callers and Interfaces: NativeToolchainRegistry object, called by VerifiedCompletionGate, ToolchainExecutionDriver, and DecoupledWorkerSynthesizer. Values and Symbols: register, getDriver, requireDriver, locateBinary, LanguageToolchainDriver interface. Imports and Exports: Imports specific language drivers for Kotlin, Python, Go, Rust, and TypeScript; exports unified toolchain controller. Implementation Directive: Provide pluggable drivers for discovering, inspecting, configuring, and invoking host SDKs and compilers. Execute builds, linter checks, and unit tests through sandboxed execution drivers without polluting global system paths. Further Research Directive: Research Hermetic Build toolchain isolation patterns (such as Nix-like environment sandboxing) for portable zero-dependency compiler execution on mobile targets.

ATOM 20.3: DEEP AST SYMBOL NAVIGATION AND REFERENCE GRAPH File Path: src/main/kotlin/atropos/ast/AstSymbolGraph.kt Callers and Interfaces: AstSymbolGraph class, called by CommandRegistry, VerifyCommand, and AstCommandHandler. Values and Symbols: AstSymbol, AstSymbolKind, buildGraph, findCallers, findDefinitions, impactedSymbolEvidence. Imports and Exports: Imports java.nio.file.Path; exports indexed symbol callgraphs and dependency maps. Implementation Directive: Index abstract syntax tree declarations across the entire repository. Support lightning-fast symbol lookups, caller discovery, and impact analysis to allow both human developers and autonomous AI agents to navigate code relationships accurately. Further Research Directive: Research incremental AST cache invalidation strategies using Git index tree hashes to update symbol graphs only for files modified in the active branch.

ATOM 20.4: EXTENSIBLE MCP TOOL SERVER LIFECYCLE GOVERNANCE File Path: src/main/kotlin/atropos/core/integration/GenericMcpHostManager.kt Callers and Interfaces: GenericMcpHostManager class, invoked by McpCommandHandler and AutonomousOrchestrator. Values and Symbols: GenericMcpServerConfig, GenericMcpToolDescriptor, startServer, listTools, callTool, shutdownServer. Imports and Exports: Imports org.json.JSONObject, java.util.concurrent.ConcurrentHashMap; exports MCP session manager. Implementation Directive: Manage Model Context Protocol (MCP) server lifecycles over standard IO and HTTP transports. Enforce strict tool execution budgets, timeout ceilings, and output payload size limits to prevent external tool integrations from hanging or exhausting host memory. Further Research Directive: Research dynamic capability negotiation and JSON-RPC

multiplexing over WebAssembly runtime boundaries for embedded MCP server sandboxing.

BATCH 5 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (SECTIONS 21 TO 25)

ATOM 21.1: UNIFIED CLIENT-ENGINE COMMAND EXECUTION PIPELINE File Path: src/main/kotlin/atropos/bridge/BridgeCommandHandler.kt Callers and Interfaces: BridgeCommandHandler class, invoked by LocalEngineBridge and BridgeRouteCatalog. Values and Symbols: execute, allowed, run, BridgeCommandOutput, PortCommandPolicy. Imports and Exports: Imports atropos.bridge.http.HttpRequest, atropos.bridge.http.HttpResponse; exports command execution responses. Implementation Directive: Route all commands initiated from Android or Web clients directly to the core engine process. Never simulate command responses locally on client surfaces. Ensure execution results, exit codes, stdout, and stderr travel back to the client interface accurately without distortion. Further Research Directive: Research binary multiplexing protocols for full-duplex interactive terminal streaming over standard WebSocket connections on mobile networks.

ATOM 21.2: ANDROID TERMINAL EMULATOR AND PTY BINDING File Path: apps/atropos-android/src/main/kotlin/atropos/android/ui/CliParityConsoleScreen.kt Callers and Interfaces: CliParityConsoleScreen composable, rendered by AndroidSovereignShell and SovereignWorkstationScreen. Values and Symbols: CliParityConsoleScreen, NativeTerminalViewport, PtyNativeBridge, spawnPty. Imports and Exports: Imports androidx.compose.runtime, atropos.core.toolchain.PtyNativeBridge; exports interactive terminal composable. Implementation Directive: Embed a real pseudoterminal (pty) session within the Android console screen. Bind input and output streams directly to the host shell environment, providing identical command execution semantics to the desktop CLI. Further Research Directive: Research Android JNI bindings for openpty and forkpty system calls across ARM64 bionic architectures to support native shell sessions without root.

ATOM 21.3: REAL-TIME ENGINE EXECUTION TELEMETRY RETURN File Path: src/main/kotlin/atropos/bridge/BridgeEventHub.kt Callers and Interfaces: BridgeEventHub object, invoked by BridgeEventsHandler and EngineHttpServer. Values and Symbols: emit, getAfter, detectChanges, Event data class, cursor, sequence, MAX_EVENTS. Imports and Exports: Imports java.time.Instant, java.util.concurrent.ConcurrentHashMap; exports real-time event sequences. Implementation Directive: Stream live execution progress, active queue states, and tool outcomes back to mobile and web clients. Ensure client interfaces display real-time engine activity without requiring manual page reloads or aggressive polling. Further Research Directive: Research Server-Sent Events (SSE) backpressure mitigation strategies to throttle high-frequency log updates gracefully when client rendering buffers lag.

ATOM 22.1: SOVEREIGN SELF-HOST GOAL EXECUTION PIPELINE

File Path: src/main/kotlin/atropos/core/agent/SelfHostGoalService.kt Callers and Interfaces: SelfHostGoalService class, called by SelfHostCommand and BridgeSelfHostHandler. Values and Symbols: startGoal, loadGoal, resumeGoal, advanceGoal, promoteVerifiedJar, SelfHostResult. Imports and Exports: Imports atropos.core.dag.DagExecutionService, atropos.core.verification.VerifiedCompletionGate; exports SelfHostResult status. Implementation Directive: Implement owner-only self-hosting, self-building, and self-repairing execution routines. Execute full-repo analysis, multi-wave DAG synthesis, and isolated worktree compilations to allow the engine to rebuild its own binaries from source. Further Research Directive: Research zero-downtime hot-swapping techniques for running JVM processes using dynamic classloader rebinding and shared memory handoff.

ATOM 22.2: CRYPTOGRAPHIC OWNER PRIVILEGE VERIFICATION File Path: src/main/kotlin/atropos/core/security/OwnerAuthorizationService.kt Callers and Interfaces: OwnerAuthorizationService class, called by Phase20Authorization and SelfHostCommand. Values and Symbols: isConfigured, verifyPassword, requireAuthorization, validateToken, AuthorizationToken. Imports and Exports: Imports javax.crypto.SecretKeyFactory, java.security.SecureRandom; exports AuthorizationToken verification models. Implementation Directive: Enforce cryptographic authentication before allowing any self-hosting or administrative operations. Ordinary authenticated users in the Supabase database must be strictly denied access to these privileged endpoints unless verified by the owner master key. Further Research Directive: Research hardware FIDO2 and WebAuthn security key integration for biometric owner authentication in headless server environments.

ATOM 22.3: PHASE 20 INSIDE-OUT SELF-BUILD GATE File Path: src/main/kotlin/atropos/core/phase20/Phase20Laws.kt Callers and Interfaces: Phase20Laws class, invoked by SelfBuildValidationRule and GovernedCompileGate. Values and Symbols: validateSourceMutationCaller, validateComplianceScore, validateSeparationOfDuties, validateExitZeroForL6. Imports and Exports: Imports atropos.core.phase20.ClaimLevel; exports governance verification assertions. Implementation Directive: Ensure that self-modifying code proposals pass strict separation-of-duties and exit-zero compilation gates. Prohibit the AI from modifying its own safety guardrails or self-verifying its mutations without independent compiler and test execution passes. Further Research Directive: Research automated Hoare-logic invariant provers to verify that candidate compiler patches mathematically preserve core security properties before execution.

ATOM 23.1: CONTINUOUS TELEMETRY HARVESTER AND RING BUFFER CONTROL File Path: src/main/kotlin/atropos/core/telemetry/TelemetryRingBuffer.kt Callers and Interfaces: TelemetryRingBuffer class, invoked by DynamicDatasetSyncService and WaveOrchestratorTelemetryWiring. Values and Symbols: offer, triggerAsyncFlush, spoolRecordDirect, listPendingSpoolFiles, purgeSpoolFile. Imports and Exports: Imports java.util.concurrent.ArrayBlockingQueue,

atropos.core.storage.StorageSupervisor; exports buffered telemetry records. Implementation Directive: Implement administrative controls allowing the owner to pause, resume, and inspect the continuous background telemetry and data scraping engine. Ensure buffers flush cleanly to disk upon shutdown without losing recorded dataset frames. Further Research Directive: Research memory-mapped circular disk buffers (ring buffers) for zero-allocation telemetry spooling during continuous 24/7 background operation.

ATOM 23.2: ADMINISTRATIVE DATASET SCRAPING SUPERVISOR INTERFACE File Path: src/main/kotlin/atropos/core/memory/NeuralMemoryDopamineSync.kt Callers and Interfaces: NeuralMemoryDopamineSync class, called by DynamicDatasetSyncService and CLI telemetry commands. Values and Symbols: recordTrajectory, pushDopamineSync, querySimilarFailureTrajectories, EpisodicMemoryRecord. Imports and Exports: Imports org.json.JSONObject, atropos.core.supremacy.OpenCoreAirGapController; exports dataset synchronization status. Implementation Directive: Provide owner-only controls to monitor and manage synchronization with external research datasets (such as Hugging Face Atropos-Dopamine). Display live operational indicators, transmission status, and sync errors truthfully. Further Research Directive: Research differential JSON-L chunk compression algorithms to minimize network egress bandwidth during high-volume dataset sync passes.

ATOM 23.3: SCRAPING OPERATIONAL RUN HISTORY AND ERROR LEDGER File Path: src/main/kotlin/atropos/core/telemetry/TelemetryBackpressureAndDiskBudgetGoverno Callers and Interfaces: TelemetryBackpressureAndDiskBudgetGovernor class, invoked by TelemetryRingBuffer. Values and Symbols: currentSpoolSizeBytes, computeAdmissionProbability, shouldAcceptRecord, isUnderBackpressure. Imports and Exports: Imports atropos.core.storage.StorageSupervisor; exports backpressure admission decisions. Implementation Directive: Maintain an auditable history of recent scraping and telemetry passes, recording timestamps, record counts, and failure diagnostics. Halt scraping automatically when local disk space drops below safety thresholds. Further Research Directive: Research Lyapunov stability models for dynamic backpressure control to prevent storage exhaustion during bursty telemetry generation.

ATOM 24.1: HUMAN-IN-THE-LOOP AGENT AUTHORIZATION CHALLENGE File Path: src/main/kotlin/atropos/core/governance/Phase20ProposalGate.kt Callers and Interfaces: Phase20ProposalGate class, called by InboundToolBridge and BoundedAgencyGate. Values and Symbols: evaluateProposal, approveProposal, ActionProposal, ProposalEvaluation, operatorSignature. Imports and Exports: Imports atropos.core.integration.McpEvidenceAndAuthorityPolicy; exports ProposalEvaluation verdicts. Implementation Directive: When an autonomous agent requires administrative authorization for a privileged action, pause execution and prompt the human owner explicitly. The agent must never assume, fabricate, or bypass the authorization token. Further Research Directive: Research cryptographic challenge-response protocols that bind approval tokens to specific action hashes and execution timeouts.

ATOM 24.2: ZERO-RETENTION PASSWORD AND CREDENTIAL PROTECTION File Path: src/main/kotlin/atropos/core/security/SecretEgressGate.kt Callers and Interfaces: SecretEgressGate object, called by ProviderChatDispatcher, TelemetrySanitizationGate, and RedactionFilter. Values and Symbols: registerCanary, scan, hasSecretLeak, scanTurn, EgressViolation, Canary. Imports and Exports: Imports java.time.Instant, atropos.core.security.SecretSinkKind; exports EgressViolation audit lists. Implementation Directive: Ensure administrative passwords, master keys, and private tokens are never logged, stored in source code, or echoed into agent prompts. Scrub sensitive tokens from all outbound model contexts and terminal output streams. Further Research Directive: Research automated regex and entropy scanning pipelines with sub-millisecond execution times for streaming token redaction.

ATOM 24.3: SECURE PBKDF2 PASSWORD VERIFICATION AND TOKEN ISSUANCE File Path: src/main/kotlin/atropos/core/phase20/Phase20Authorization.kt Callers and Interfaces: Phase20Authorization class, called by SelfHostCommand and OwnerAuthorizationService. Values and Symbols: setup, verifyPassword, requireAuthorization, validateToken, pbkdf2, generateSalt, AuthorizationToken. Imports and Exports: Imports javax.crypto.spec.PBEKeySpec, java.security.SecureRandom; exports ephemeral AuthorizationToken objects. Implementation Directive: Hash owner passwords using PBKDF2-HMAC-SHA256 with cryptographically random salts and high iteration counts. Issue short-lived, cryptographically signed authorization tokens that expire automatically after task completion. Further Research Directive: Research memory-safe Argon2id password hashing implementations for JVM environments to provide maximum resistance against GPU-assisted brute-force attacks.

ATOM 25.1: HIGH-DENSITY ANDROID CYBER-HUD THEME SYSTEM File Path: app/src/main/java/com/atropos/android/app/theme/AtroposCyberTheme.kt Callers and Interfaces: AtroposCyberTheme composable, used across all Android app screens and dialogs. Values and Symbols: CyberHudColors, LocalCyberHudColors, cyberGlass, neonPulseBorder, cyberGridBackground. Imports and Exports: Imports androidx.compose.material3.darkColorScheme, androidx.compose.ui.graphics.Color; exports global theme composition locals. Implementation Directive: Provide a high-density, dark-mode HUD theme optimized for technical mobile workflows. Deliver clean high-contrast syntax highlighting, refined panel borders, and organized visual hierarchy to minimize eye fatigue during intensive development sessions. Further Research Directive: Research dynamic OLED black-pixel battery optimization techniques in Jetpack Compose to reduce device power consumption on mobile screens.

ATOM 25.2: HARDWARE-ACCELERATED SVG AND MOTION GRAPHICS ENGINE File Path: apps/atropos-android/src/main/kotlin/atropos/android/ui/AtroposHudTheme.kt Callers and Interfaces: SixAnswersRadarCard, NativeTerminalViewport, AtroposHudTheme, called by AndroidSovereignShell. Values and Symbols: AtroposColorPalette, EngineSixAnswers, EvidenceRef, SixAnswer-

sRadarCard, SovereignEvidenceDrawer. Imports and Exports: Imports androidx.compose.foundation.Canvas, androidx.compose.ui.graphics; exports custom vector graphics composables. Implementation Directive: Render radar status diagrams, DAG graphs, and telemetry meters using hardware-accelerated Compose Canvas draw calls and optimized vector graphics. Ensure visual elements enhance comprehension without causing frame drops or visual noise. Further Research Directive: Research path morphing vector animation engines in Compose for smooth transitions between graph layouts and radar status displays.

ATOM 25.3: ACCESSIBLE TOUCH DENSITY AND NON-COLOR STATE INDICATORS File Path: app/src/main/java/com/atropos/android/app/ui/OneHandDensity.kt Callers and Interfaces: OneHandDensity class, called by MobileComposer, CheckpointChip, and SixAnswersPanel. Values and Symbols: touchTargetDp, ReachZone enum, OfflineResume data class, reachZone, offlineResume. Imports and Exports: Imports standard Compose geometry units; exports touch target bounds and ergonomic reach zones. Implementation Directive: Enforce minimum 44dp touch target bounds across all interactive controls while maintaining high visual density. Accompany all color-coded status badges with distinct geometric glyphs and text labels to satisfy accessibility standards. Further Research Directive: Research thumb-zone ergonomic mapping algorithms that dynamically position primary action controls within comfortable one-handed reach on large mobile screens.

BATCH 6 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (5 MAJOR SECTIONS FROM V2)

ATOM M1.1: ACTIVITY BAR AND PRIMARY VIEW CONTAINER SWITCHER File Path: apps/web/src/components/app-shell/app-sidebar.tsx Callers and Interfaces: AppSidebar component, invoked by AppShell and WorkbenchMain layouts. Values and Symbols: NavLinks, useNavItems, navigationSpine, activeProjectSection, SectionAccent. Imports and Exports: Imports NavLinks, useNavItems; exports AppSidebar navigation activity bar. Implementation Directive: Implement a persistent left-edge Activity Bar replicating modern IDE view switching. Provide dedicated icon triggers for Explorer, Global Search, Version Control, Debug Console, Extensions, and Settings with instant visual active-state indicators. Further Research Directive: Research keyboard navigation accesskey standards and ARIA tablist patterns for rapid keyboard-only activity bar tab switching.

ATOM M1.2: EXPLORER HIERARCHICAL FILE PANE File Path: apps/web/src/components/ide/workspace-file-tree.tsx Callers and Interfaces: WorkspaceFileTree component, rendered within SovereignIdeWorkstation and WorkbenchShell. Values and Symbols: FileTreeItem, FileNode, rootItems, selectedPath, onSelectFile, onRefresh, FileTreeNodeRow. Imports and Exports: Imports Lucide icon components; exports WorkspaceFileTree interactive directory tree. Implementation Directive: Build a dense, collapsible Explorer pane displaying nested project directories, source files, and asset folders.

Support inline file creation, directory collapsing, and visual badges for modified, added, or untracked files. Further Research Directive: Research virtualized DOM tree rendering with dynamic node height calculation for rendering multi-thousand-file repositories at 60 FPS.

ATOM M1.3: EDITOR GROUP WITH SPLIT VIEWS AND TABS File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor component, called by SovereignIdeWorkstation and EditorTabs. Values and Symbols: MonacoCodeEditorProps, filePath, initialContent, language, readOnly, isDirty, onSave. Imports and Exports: Imports React hooks, Lucide icons; exports MonacoCodeEditor container. Implementation Directive: Replicate modern editor tab groups supporting multiple open documents, split views (horizontal and vertical), breadcrumb paths, and tab reordering. Track unsaved dirty state with visual indicator dots. Further Research Directive: Research shared memory buffers in browser-based Monaco editor instances to enable instant zero-copy diff comparison between editor tabs.

ATOM M1.4: COLLAPSIBLE BOTTOM PANEL FOR TERMINAL, OUTPUT, AND PROBLEMS File Path: apps/web/src/components/workbench/logpanel.tsx Callers and Interfaces: LogPanel component, embedded in WorkbenchShell bottom panel container. Values and Symbols: LogPanel, subscribeActivity, ActivityEvent, EventActivityEvent, filterByStage. Imports and Exports: Imports event streaming client utilities; exports LogPanel console component. Implementation Directive: Implement an extensible, collapsible bottom panel with dedicated tabs for Integrated Terminal, Build Output, Problems, Debug Console, and Real-Time Activity Streams. Allow toggling height and maximizing to full screen. Further Research Directive: Research terminal multiplexing state preservation in React state managers when collapsible bottom panels are hidden and restored.

ATOM M1.5: HIGH-CONTRAST SEMANTIC SYNTAX HIGHLIGHTING AND UNIFIED TYPOGRAPHY File Path: app/src/main/java/com/atropos/android/app/theme/AtroposTyp
Callers and Interfaces: AtroposTypography object, used across Android Compose screens and Web Monaco CSS styles. Values and Symbols: monoCode, monoGutter, monoToken, commandPrompt, hudHeader, hudBody, badgeLabel. Imports and Exports: Imports androidx.compose.ui.text.TextStyle; exports standardized typographic scale tokens. Implementation Directive: Enforce an identical semantic syntax highlighting color palette and monospaced typography across Web, Android, and CLI surfaces. Ensure high-contrast tokens for keywords, types, strings, and comments to reduce cognitive eye strain. Further Research Directive: Research TextMate grammar to Tree-sitter highlighter mappings to achieve identical semantic token coloring between desktop Monaco and mobile syntax renderers.

ATOM M2.1: INDEPENDENT LOCAL IDE WEB SERVER SUBSTRATE File Path: src/main/kotlin/atropos/bridge/http/EngineHttpServer.kt Callers and Interfaces: EngineHttpServer class, instantiated by LocalEngineBridge and AtroposBridge. Values and Symbols: EngineHttpServer, routeTable, bound-

Port, sseClients, broadcastEvent, isRunning. Imports and Exports: Imports atropos.core.security.RedactionFilter; exports running embedded HTTP server. Implementation Directive: Operate the local IDE web server completely detached from external cloud routing assumptions. Bind to dynamically allocated local ports, serve the web workbench bundle, and expose REST and WebSocket endpoints for full local development. Further Research Directive: Research zero-dependency native HTTP/2 implementation in pure Kotlin to multiplex REST and SSE over single persistent TCP connections.

ATOM M2.2: EPHEMERAL MOCK DATA ENGINE INITIALIZATION File Path: src/main/kotlin/atropos/core/welcome/WelcomeArtifact.kt Callers and Interfaces: WelcomeArtifact class, invoked by WelcomePanel and HomeStateProvider. Values and Symbols: WelcomeArtifact, render, contentId, welcomeMessage, demoProjectStructure. Imports and Exports: Imports java.security.MessageDigest; exports immutable WelcomeArtifact descriptors. Implementation Directive: When initialized without an existing workspace, synthesize clean mock project descriptors and file trees. Prevent the local server from leaking host machine files, personal paths, or developer configurations into the fresh interface. Further Research Directive: Research deterministic pseudorandom seed generation for creating realistic synthetic multi-language project structures for testing onboarding UX.

ATOM M2.3: CYCLICAL CODEBASE DAG VISUALIZER AND IMPORT-EXPORT MAPPER File Path: apps/web/src/components/graph/graph-workspace.tsx Callers and Interfaces: GraphWorkspace component, rendered on project graph routes and developer tools views. Values and Symbols: GraphWorkspace, ReactFlowProvider, semanticGraphToRendererContent, filterGraphContent. Imports and Exports: Imports xyflow/react, graph layout clients; exports GraphWorkspace interactive canvas. Implementation Directive: Visualize codebase dependencies, imports, exports, and data flow as an interactive directed acyclic graph. Highlight cyclic dependency risks, identify orphan modules, and map relations between interfaces and implementations. Further Research Directive: Research WebGL and Canvas2D hybrid graph layout engines capable of smoothly rendering graphs with tens of thousands of nodes and edges.

ATOM M2.4: FULL CRUD DISK MUTATION AND COMMIT PIPELINE File Path: apps/web/src/lib/workspace/client.ts Callers and Interfaces: writeWorkspaceFile, checkTerritoryMembership, called by EditorTabs and SovereignIdeWorkstation. Values and Symbols: writeWorkspaceFile, WorkspaceWriteResult, bytesWritten, sha256, checkTerritoryMembership. Imports and Exports: Imports readEngine, writeEngine; exports disk mutation API handlers. Implementation Directive: Support full create, read, update, and delete operations on physical workspace files within permitted territory boundaries. Enable direct Git staging and atomic commit generation that pass local CI checks. Further Research Directive: Research optimistic local file locking mechanisms with conflict resolution journals to handle concurrent edits

between human operators and autonomous agents.

ATOM M2.5: DENSE HIERARCHICAL WORKSPACE AND USER SETTINGS ENGINE File Path: src/main/kotlin/atropos/core/config/AtroposUnifiedConfig.kt Callers and Interfaces: AtroposUnifiedConfig class, called by CliModalStateMachine and SettingsMatrixPage. Values and Symbols: AtroposUnifiedConfig, defaultValues, configFile, get, set, cycleOption, renderMatrix. Imports and Exports: Imports atropos.core.AtroposRepoRootLocator; exports global configuration manager. Implementation Directive: Deliver an exceptionally dense, hierarchical settings architecture supporting global user preferences, project-level workspace overrides, direct JSON configuration editing, keybinding customization, and MCP server declarations. Further Research Directive: Research JSON Schema draft-07 validation integration to validate settings JSON edits live with autocomplete and tooltip schema documentation in Monaco.

ATOM M2.6: SUPABASE AUTHENTICATION AND DASHBOARD SCREEN GATEKEEPER File Path: apps/web/src/components/auth/sign-in-form.tsx Callers and Interfaces: SignInForm component, rendered on auth sign-in and account creation routes. Values and Symbols: SignInForm, createSupabaseBrowserClient, sanitizeReturnPath, email, password. Imports and Exports: Imports react-hook-form, zod; exports interactive authentication form. Implementation Directive: Enforce Supabase authentication for cloud synchronization and administrative features. Upon authentication, validate user authorization levels and grant access to hidden internal telemetry, evaluation, and governance dashboard screens. Further Research Directive: Research Row-Level Security (RLS) policies in PostgreSQL to restrict project telemetry data visibility strictly to authenticated team members.

ATOM M3.1: TOUCH-OPTIMIZED ARM64 IDE PARITY ENGINE File Path: apps/atropos-android/src/main/kotlin/atropos/android/ui/SovereignWorkstationScreen.kt Callers and Interfaces: SovereignWorkstationScreen composable, rendered as the primary Android IDE container. Values and Symbols: SovereignWorkstationScreen, MobileSplitEditor, HapticApprovalBanner, ApprovalItem. Imports and Exports: Imports androidx.compose.material3, Compose layout primitives; exports mobile IDE screen. Implementation Directive: Optimize the complete IDE feature set for ARM64 Android devices with touch interfaces. Provide responsive split panes, haptic confirmation feedback, and adaptive on-screen keyboard spacing. Further Research Directive: Research Android RenderThread frame budget profiling to ensure touch drag interactions maintain strict 16ms frame times under sustained CPU workload.

ATOM M3.2: CROSS-SURFACE PREFERENCE AND SESSION TOKEN SYNC File Path: apps/web/src/lib/contexts/session-state-context.tsx Callers and Interfaces: SessionStateProvider, useSessionState, wraps primary Web and Android client roots. Values and Symbols: SessionState, activeProjectId, openTabs, simpleModeEnabled, developerToolsEnabled, informationLevel. Imports and Exports: Imports restoreSession utility; exports session state

context and reactive hooks. Implementation Directive: Synchronize active project tabs, layout themes, disclosure levels, and authentication tokens across Android, Web, and desktop CLI sessions via Supabase and local cache persistence. Further Research Directive: Research Conflict-Free Replicated Data Types (CRDTs) for synchronizing multi-device editor cursor positions and active tabs in real time.

ATOM M3.3: NON-INTRUSIVE BOTTOM SHEET TERMINAL OVERLAYS File Path: app/src/main/java/com/atropos/android/app/ui/ToolsTimelineSheet.kt Callers and Interfaces: ToolsTimelineSheet composable, invoked by floating action button in mobile IDE. Values and Symbols: ToolsTimelineSheet, sheetState, ToolsSection, TimelineSection, FileTreeSection, ThinkingSection. Imports and Exports: Imports androidx.compose.material3.ModalBottomSheet; exports overlay tools sheet. Implementation Directive: House complex terminal outputs, diagnostic logs, and multi-step tool timelines inside gesture-driven bottom sheet overlays. Prevent screen clutter on mobile viewports while preserving complete terminal execution capabilities. Further Research Directive: Research ModalBottomSheet drag-handle velocity physics to enable smooth half-expanded and fully-expanded panel anchoring on Android.

ATOM M4.1: HARDWARE-AGNOSTIC STANDALONE JAR RELEASE ENGINE File Path: src/main/kotlin/atropos/core/BuildStamp.kt Callers and Interfaces: BuildStamp object, invoked by CLI banner, self-update routines, and version APIs. Values and Symbols: BuildStamp, line, version, commit, artifactSha256, MessageDigest. Imports and Exports: Imports java.security.MessageDigest, java.nio.file.Path; exports build provenance string. Implementation Directive: Package the complete Atropos core engine, CLI, and local web server into a single hardware-agnostic standalone JAR. Ensure any user can download the artifact via curl or wget and run it instantly on any POSIX or Windows system with a standard JVM. Further Research Directive: Research GraalVM Native Image ahead-of-time compilation for generating standalone zero-dependency native binaries for Linux, macOS, and Windows.

ATOM M4.2: DYNAMIC EPHEMERAL LOCALHOST PORT BINDING SUBSYSTEM File Path: src/main/kotlin/atropos/bridge/LocalEngineBridge.kt Callers and Interfaces: LocalEngineBridge object, called by interactive runner and background daemon launchers. Values and Symbols: LocalEngineBridge, PORT_VARIABLE, fromEnvironment, server, ServerSocket. Imports and Exports: Imports atropos.bridge.http.EngineHttpServer; exports running server handle with allocated port. Implementation Directive: Bind the local web server to dynamically assigned, ephemeral localhost ports when default ports are occupied. Pass the resolved port back to the client launcher without hardcoding absolute paths or ports. Further Research Directive: Research SO_REUSEADDR and SO_REUSEPORT socket flags across operating systems to handle rapid server restarts without port binding timeouts.

ATOM M4.3: TREE-SITTER AST MULTI-WAVE DAG AND DOPAMINE DATASET SYNC File Path: src/main/kotlin/atropos/core/telemetry/WaveOrchestratorTelemetryWiring.kt

Callers and Interfaces: WaveOrchestratorTelemetryWiring class, invoked by AutonomousWaveOrchestrator. Values and Symbols: OrchestrationWave enum, onWaveTransition, recordThoughtStep, pushDopamineSync. Imports and Exports: Imports atropos.core.parser.TreeSitterGrammarBridge, NeuralMemoryDopamineSync; exports telemetry pipeline. Implementation Directive: Coordinate multi-wave contract synthesis DAGs using Tree-sitter AST parsing. Continuously stream execution telemetry, compiler diagnostics, and repair trajectories to the Hugging Face Atropos-Dopamine dataset for offline process supervision. Further Research Directive: Research Tree-sitter incremental re-parsing over byte buffers to update AST syntax nodes in microseconds during streaming code generation.

ATOM M5.1: GLOBALLY INDEXED VERCEL MARKETING PLATFORM File Path: apps/web/src/components/landing/landing-hero.tsx Callers and Interfaces: LandingHero component, rendered on public root landing page routes. Values and Symbols: LandingHero, FEATURES, UploadIcon, EvidenceIcon, GraphIcon, VerifiedIcon. Imports and Exports: Imports next link, UI cards; exports public marketing funnel hero component. Implementation Directive: Deploy a globally indexed, SEO-optimized marketing website on Vercel separate from local IDE instances. Feature compelling sections for About Atropos, SpecGraph Foundry, Data Center hardware specs, and competitive benchmarks against existing AI coding agents. Further Research Directive: Research OpenGraph image generation at the edge using Next.js ImageResponse to dynamically generate social preview cards for documentation pages.

ATOM M5.2: GITHUB DOCS-STRUCTURED DOCUMENTATION PORTAL File Path: apps/web/src/components/navigation/routes.ts Callers and Interfaces: globalRoutes, navigationSpine, projectSections, called by documentation layouts and site header. Values and Symbols: globalRoutes, devTools, specGraph, projectSections, SpineItem. Imports and Exports: Imports Next Route types; exports global route definitions. Implementation Directive: Build a comprehensive documentation portal mirroring GitHub Docs. Provide step-by-step guides, CLI command syntax references, engine deployment instructions, architecture guidelines, and video tutorials. Further Research Directive: Research Algolia DocSearch integration versus local edge vector search for instant full-text search across documentation pages.

ATOM M5.3: SUPABASE-AUTHENTICATED CLOUD PROJECT AND BILLING PORTAL File Path: apps/web/src/components/projects/project-directory.tsx Callers and Interfaces: ProjectDirectory component, rendered on authenticated user dashboard routes. Values and Symbols: ProjectDirectory, useProjectsPage, ProjectCard, ProjectPagination, DataGrid. Imports and Exports: Imports project queries; exports project management directory component. Implementation Directive: Provide a cloud-synchronized project management dashboard authenticated via Supabase. Allow developers to manage cloud projects, inspect API quotas, configure billing tiers, and provision integrations before launching local engine sessions. Further Research Directive:

Research Stripe Billing webhook integration with Supabase Edge Functions for automated real-time quota upgrades upon subscription renewal.

BATCH 7 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (CORE EDITOR AND INTERFACE SETTINGS)

ATOM C1: EDITOR FORMAT ON SAVE AUTOMATION File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor component, invoked by onSave handler in SovereignIdeWorkstation. Values and Symbols: editor.formatOnSave boolean configuration, formatDocumentAction, runFormatterOnSave, isDirty. Imports and Exports: Imports monaco editor formatting providers; exports formatted buffer string before disk write. Implementation Directive: Intercept editor save commands and run registered Prettier, Ktlint, or language-specific AST formatters over the buffer. Ensure syntax errors do not corrupt the file and preserve cursor offsets during text replacement. Further Research Directive: Research incremental AST range formatting to format only touched line ranges rather than entire multi-thousand-line documents on save.

ATOM C2: EDITOR LINE WORD WRAPPING CONTROLLER File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor options configuration, called by SettingsMatrixPage and ViewportLayout. Values and Symbols: editor.wordWrap configuration, wordWrapColumn, wordWrapMinified, wrappingIndent. Imports and Exports: Imports editor options interfaces; exports reactive word-wrap state. Implementation Directive: Toggle soft word-wrapping based on user configuration and viewport width. Break long lines cleanly at word boundaries or indent columns without injecting newline characters into physical source files. Further Research Directive: Research GPU-accelerated glyph layout measurement algorithms in web workers for calculating wrapping breakpoints on ultra-wide lines.

ATOM C3: EDITOR OVERVIEW MINIMAP COMPONENT File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor minimap options, rendered in editor canvas gutter. Values and Symbols: editor.minimap.enabled boolean, minimapScale, showSlider, renderCharacters. Imports and Exports: Imports monaco minimap configuration types; exports minimap viewport overlay. Implementation Directive: Render a scaled visual overview of the entire file along the editor right edge. Highlight active viewport bounds, search matches, and error markers, allowing smooth drag-to-scroll navigation. Further Research Directive: Research canvas downsampling and pixel caching techniques to render minimaps of large files with zero frame drops during fast scrolling.

ATOM C4: BRACKET PAIR COLORIZATION ENGINE File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor syntax engine, called during buffer tokenization passes. Values and Symbols: editor.bracketPairColorization.enabled boolean, bracketPairGuides, bracketColors palette. Imports and Exports: Imports tokenization

options; exports colorized syntax tokens to renderer. Implementation Directive: Color-code matching nested parentheses, brackets, and braces using distinct cyclic palette colors. Render vertical indent guide lines connecting matching pairs to clarify deeply nested logic blocks. Further Research Directive: Research Tree-sitter bracket matching performance versus algorithmic stack-based scanning for instant color updates during live typing.

ATOM C5: STICKY SCROLL SIGNATURE PINNING SUBSYSTEM File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor scroll listener, rendered at the top of the editor viewport. Values and Symbols: editor.stickyScroll.enabled boolean, maxStickyLines integer, stickyScopeModel. Imports and Exports: Imports AST symbol scope tree; exports pinned header widget DOM nodes. Implementation Directive: Pin enclosing class, interface, and function signature declarations to the top of the editor as the user scrolls through long implementations. Update pinned headers instantly as scope boundaries change. Further Research Directive: Research fast scope tree lookups using binary interval search trees to determine sticky scopes in sub-millisecond execution times.

ATOM C6: MULTI-CURSOR MODIFIER BINDING CONTROLLER File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor mouse down and keyboard listener, called by editor interaction bus. Values and Symbols: editor.multiCursorModifier enum (alt or ctrlCmd), addCursorAtPosition, selectionList. Imports and Exports: Imports mouse event handlers; exports active cursor position collections. Implementation Directive: Allow users to place multiple simultaneous editing cursors using the configured modifier key and mouse click. Support simultaneous typing, backspacing, and multi-line selection box expansion. Further Research Directive: Research transactional multi-cursor text edit merging algorithms to prevent cursor collision and invalid offset offsets during simultaneous paste operations.

ATOM C7: SNIPPET AUTOCOMPLETE SUGGESTION PRIORITIZATION File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor completion item provider, called by autocomplete dropdown engine. Values and Symbols: editor.snippetSuggestions enum (top, bottom, inline, none), snippetCompletionProvider. Imports and Exports: Imports completion item models; exports ranked completion list. Implementation Directive: Control the ranking of reusable code snippets within the IntelliSense popup. Position snippets above or below symbol completions according to user preference, expanding placeholders via Tab key navigation. Further Research Directive: Research context-aware completion ranking models that dynamically promote snippets based on AST enclosing scope and recently used templates.

ATOM C8: AUTO-CLOSING BRACKETS AND QUOTES ENGINE File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor character insertion listener, invoked on keystroke events. Values and Symbols: editor.autoClosingBrackets enum (always, languageDe-

fined, beforeWhitespace, never), autoClosingQuotes. Imports and Exports: Imports language configuration service; exports modified text buffer edits. Implementation Directive: Automatically insert matching closing symbols when typing opening brackets or quotation marks. Skip over existing closing symbols when typed directly in front of them to avoid duplicate characters. Further Research Directive: Research language-aware quote closure rules that detect raw string literals, docstrings, and triple-quote blocks to prevent unwanted character insertions.

ATOM C9: SIMULTANEOUS LINKED EDITING FOR TAG PAIRS File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor linked editing range provider, called on XML/HTML token selection. Values and Symbols: editor.linkedEditing boolean, provideLinkedEditingRanges, linkedRanges list. Imports and Exports: Imports DOM/HTML parser types; exports synchronized multi-range edit targets. Implementation Directive: Automatically update matching opening and closing HTML or XML tags simultaneously when either tag name is modified. Highlight paired tag ranges visually during editing. Further Research Directive: Research incremental XML/JSX CST parsing to track matching tag pairs across asynchronous file modifications without blocking the UI thread.

ATOM C10: INVISIBLE WHITESPACE RENDERING SUBSYSTEM File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor glyph renderer, called during text line layout passes. Values and Symbols: editor.renderWhitespace enum (none, boundary, selection, trailing, all), whitespaceGlyphPalette. Imports and Exports: Imports editor view options; exports whitespace glyph draw calls to canvas. Implementation Directive: Render subtle glyph dots for spaces and arrows for tabs based on user settings. Make trailing spaces and mixed indentation formats immediately visible to maintain clean code standards. Further Research Directive: Research low-overhead glyph rendering in HTML5 Canvas using pre-computed sprite atlases for whitespace symbols.

ATOM C11: INDENTATION TAB SIZE SPECIFICATION CONTROLLER File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor model options, called when files open or language changes. Values and Symbols: editor.tabSize integer (default 2 or 4), detectIndentation, updateModelOptions. Imports and Exports: Imports model configuration interfaces; exports active tab stop measurements. Implementation Directive: Enforce the exact number of spaces represented by a single tab indentation level. Automatically infer tab size from existing file contents if auto-detection is enabled, or fallback to workspace defaults. Further Research Directive: Research statistical indentation detection heuristics across large codebases to accurately identify project formatting conventions from sample lines.

ATOM C12: TAB-TO-SPACES CONVERSION ENGINE File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor keyboard action handler, invoked on Tab and

Backspace keypresses. Values and Symbols: editor.insertSpaces boolean, convertTabsToSpacesAction, softTabDelete. Imports and Exports: Imports editor edit operations; exports transformed whitespace strings. Implementation Directive: Convert Tab key presses into the configured number of space characters when insertSpaces is true. Emulate tab stop deletion on backspace so removing indentation behaves like single-character deletion. Further Research Directive: Research mixed-indentation normalization pipelines that convert legacy hard-tab files into modern spaces upon save without creating unreadable git diffs.

ATOM C13: SMOOTH CARET ANIMATION AND BLINK TRANSITIONS File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor cursor renderer, called on cursor position update ticks. Values and Symbols: editor.cursorSmoothCaretAnimation enum (off, explicit, on), cursorBlinking enum. Imports and Exports: Imports cursor animation CSS styles; exports animated caret element styles. Implementation Directive: Animate cursor movement fluidly between character positions using CSS transitions and spring physics. Support various blinking behaviors (blink, smooth, phase, expand, solid) to enhance visual comfort. Further Research Directive: Research GPU-synchronized requestAnimationFrame loops for zero-lag cursor interpolation matching high-refresh-rate 120Hz displays.

ATOM C14: PROGRAMMING FONT LIGATURES INTEGRATION File Path: app/src/main/java/com/atropos/android/app/theme/AtroposTypography.kt Callers and Interfaces: AtroposTypography object, loaded by Monaco CSS declarations and Jetpack Compose typography styles. Values and Symbols: editor.fontLigatures boolean, fontFeatureSettings string (”liga”, ”calt”), monoCode. Imports and Exports: Imports OpenType font feature definitions; exports styled typography themes. Implementation Directive: Enable OpenType font ligatures for programming fonts like Fira Code and JetBrains Mono. Render combined glyphs for multi-character operators (such as arrows, comparison symbols, and equality operators) cleanly. Further Research Directive: Research web font loading performance and CSS font-display swap strategies to eliminate flash of unstyled text (FOUT) during font initialization.

ATOM C15: CONTEXTUAL CODELENS ACTION DISPLAY PROVIDER File Path: apps/web/src/components/ide/ast-visualizer.tsx Callers and Interfaces: AstVisualizer component, called by Monaco CodeLens provider interface. Values and Symbols: editor.codeLens boolean, provideCodeLenses, codeLensCommand, referenceCount. Imports and Exports: Imports AST symbol hierarchy, test runner interfaces; exports actionable CodeLens items. Implementation Directive: Display actionable metadata directly above class, method, and contract definitions. Expose direct click actions to run tests, inspect callers, view Git blame, or view formal verification proofs. Further Research Directive: Research asynchronous CodeLens computation caching with cancellation tokens to prevent stale reference counts while editing code.

ATOM C16: GLOBAL SYNTAX HIGHLIGHTING AND THEME SELEC-

TOR File Path: apps/web/src/components/settings/theme-customizer.tsx Callers and Interfaces: ThemeCustomizer component, called by SettingsScreen and AppProviders. Values and Symbols: workbench.colorTheme string, applyThemeCustomization, saveThemeCustomization, CyberThemeConfig. Imports and Exports: Imports design token theme palettes; exports global theme CSS variables and Monaco theme definitions. Implementation Directive: Manage the global IDE visual theme. Provide instant preview and selection across dark, light, high-contrast, and custom cyber-HUD palettes, updating Monaco editor tokens and application chrome simultaneously. Further Research Directive: Research CSS custom property scoping and dynamic Monaco theme definition updates without full editor instance reconstruction.

ATOM C17: FILE AND FOLDER ICON THEME ENGINE File Path: apps/web/src/components/ide/workspace-file-tree.tsx Callers and Interfaces: WorkspaceFileTree component, rendered in Explorer pane file rows. Values and Symbols: workbench.iconTheme string, resolveFileIcon, resolveFolderIcon, fileIconMap. Imports and Exports: Imports SVG icon definitions; exports icon visual elements. Implementation Directive: Map file extensions, specific filenames (.gitignore, Dockerfile), and directory names to dedicated SVG visual icons. Display distinct expanded and collapsed folder icons to maximize tree readability. Further Research Directive: Research SVG sprite bundling and memoized icon resolution caches to prevent re-computation during rapid file tree scrolling.

ATOM C18: PRIMARY EXPLORER SIDEBAR LOCATION SWITCHER File Path: apps/web/src/components/layout/split-pane.tsx Callers and Interfaces: SplitPane component, invoked by WorkbenchShell and AppShell. Values and Symbols: workbench.sideBar.location enum (left, right), defaultRightWidth, minRightWidth, splitOrientation. Imports and Exports: Imports React layout primitives; exports repositionable split pane container. Implementation Directive: Allow operators to position the primary Activity Bar and Explorer sidebar on either the left or right side of the workspace. Prevent editor code shift and adjust split dividers dynamically upon repositioning. Further Research Directive: Research CSS flexbox direction inversion with persistent divider coordinate re-mapping across left-to-right (LTR) and right-to-left (RTL) locales.

ATOM C19: EDITOR TAB SIZING BEHAVIOR CONTROLLER File Path: apps/web/src/components/workbench/editor-tabs.tsx Callers and Interfaces: EditorTabs component, called by tab bar layout container. Values and Symbols: workbench.editor.tabSizing enum (fit, shrink, fixed), tabMaxWidth, closeButtonVisibility. Imports and Exports: Imports tab context store; exports styled editor tab bar items. Implementation Directive: Control editor tab width behavior across crowded workspaces. Support fixed-width tabs, proportional shrink-to-fit sizing, and dynamic horizontal scrolling with mouse wheel and drag-to-reorder capabilities. Further Research Directive: Research CSS subgrid horizontal overflow clipping with smooth scroll-into-view behavior when new

tabs are opened beyond visible viewport width.

ATOM C20: INTEGRATED PANEL DEFAULT LOCATION SELECTOR File Path: apps/web/src/components/workbench/workbench-shell.tsx Callers and Interfaces: WorkbenchShell layout coordinator, called by AppLayout and ViewportLayout. Values and Symbols: workbench.panel.defaultLocation enum (bottom, left, right), panelSize, isPanelMaximized. Imports and Exports: Imports layout slots; exports configurable multi-pane workbench layout. Implementation Directive: Allow operators to dock the bottom panel (housing terminal, output logs, and debug consoles) along the bottom, left, or right edge of the workspace. Persist panel split ratios and dock preferences across application reloads. Further Research Directive: Research responsive dock repositioning that automatically shifts vertical panels to bottom sheets when viewport width drops below tablet thresholds.

BATCH 8 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (WORKSPACE AND FILE MANAGEMENT SETTINGS)

ATOM W1: FILE AUTO-SAVE ENGINE AND TRIGGER POLICY File Path: apps/web/src/lib/editor/tabs.ts Callers and Interfaces: MonacoCodeEditor component, invoked by tab store state manager in WorkbenchTabsContext. Values and Symbols: files.autoSave enum (off, afterDelay, onFocusChange, onWindowChange), autoSaveDelay integer, markClean, setContent. Imports and Exports: Imports tab state primitives; exports automated save dispatch events to local workspace disk writers. Implementation Directive: Implement configurable auto-save triggers including debounced time delays, editor focus loss, and window blur events. Automatically commit dirty buffer contents to disk and clear dirty state flags without interrupting user typing flow. Further Research Directive: Research atomic file write streams using write-replace tempfile rename patterns to guarantee zero data loss if power fails during an auto-save cycle.

ATOM W2: EXPLORER AND SEARCH EXCLUSION FILTER File Path: src/main/kotlin/atropos/core/security/ContextPathExclusions.kt Callers and Interfaces: ContextPathExclusions object, called by FileExplorer, WorkspaceFileTree, and AgentContextCollector. Values and Symbols: files.exclude map, isExcluded, excludedDirectoryNames set, hiddenPatterns list. Imports and Exports: Imports java.nio.file.Path; exports boolean path exclusion evaluation decisions. Implementation Directive: Hide specified file glob patterns and directories from the Explorer tree and search indexes. Allow operators to configure custom exclusion patterns while ensuring critical project configuration files (.gitignore, .github) remain explicitly unblocked. Further Research Directive: Research glob-to-regex compilation caching techniques to evaluate deep directory hierarchies against exclusion lists in microsecond timeframes.

ATOM W3: FILE SYSTEM WATCHER EXCLUSION SUBSYSTEM File Path: src/main/kotlin/atropos/core/agent/AgentRunRepoStatus.kt Callers and Interfaces: AgentRunRepoStatus class, invoked by Caching-

GitWorkspaceInspector and LocalWorkQueue. Values and Symbols: files.watcherExclude map, isStageableChange, runStatus, captureResult, sensitiveEnvironmentKeys. Imports and Exports: Imports java.nio.file.Path, atropos.core.policy.BoundedProcessRunner; exports filtered filesystem status. Implementation Directive: Exclude high-churn directories such as node_modules, build caches, and .git object trees from live filesystem watchers. Prevent event queues from overflowing and eliminate CPU spikes caused by background build artifact creation. Further Research Directive: Research Linux inotify and macOS FSEvents kernel event filter flags to drop ignored directory trees at the operating system level.

ATOM W4: PERSISTENT HOT EXIT AND DIRTY BUFFER RESTORATION File Path: src/main/kotlin/atropos/core/recovery/RestartCoordinator.kt Callers and Interfaces: RestartCoordinator class, called by CrashRecoveryService, SessionStateProvider, and CliCommandHandler. Values and Symbols: files.hotExit enum (off, onExitAndWindowClose), snapshot, recoverAndSnapshot, StateSnapshot. Imports and Exports: Imports java.nio.file.Path, atropos.core.security.RedactionFilter; exports recovered buffer snapshots. Implementation Directive: Preserve unsaved dirty editor buffers, open tab sequences, and active cursor locations across application exits and unexpected crashes. Automatically restore dirty state upon launch without prompting the user to save before exit. Further Research Directive: Research encrypted local IndexedDB and write-ahead log (WAL) storage for persisting in-flight editor drafts securely without writing uncommitted code to the main Git repository.

ATOM W5: TRAILING WHITESPACE TRIMMING ON SAVE File Path: apps/web/src/lib/workspace/client.ts Callers and Interfaces: writeWorkspaceFile function, invoked by MonacoCodeEditor onSave pipeline. Values and Symbols: files.trimTrailingWhitespace boolean, trimTrailingWhitespaceAction, bytesWritten, sha256. Imports and Exports: Imports readEngine, writeEngine; exports sanitized and trimmed file contents to disk. Implementation Directive: Strip trailing space and tab characters from the ends of modified lines upon saving. Preserve trailing whitespace within active multi-line string literals or Markdown code blocks where spaces carry semantic formatting value. Further Research Directive: Research AST-guided whitespace trimming algorithms that inspect language token boundaries to avoid stripping significant whitespace in indentation-sensitive languages like Python.

ATOM W6: AUTOMATED FINAL NEWLINE ENFORCEMENT File Path: src/main/kotlin/atropos/core/agent/AgentPatchDiffNormalizer.kt Callers and Interfaces: AgentPatchDiffNormalizer class, called by AgentPatchStore and WorkspaceWriteResult. Values and Symbols: files.insertFinalNewline boolean, normalize, isContextlessAddOnlyPatch, buildAppendPatch. Imports and Exports: Imports java.nio.charset.StandardCharsets; exports normalized file content strings. Implementation Directive: Ensure all saved files terminate with a single standard newline character. Prevent Git warnings regarding missing newlines at end-of-file and eliminate noisy one-line diffs during subsequent commits.

Further Research Directive: Research POSIX line termination standard adherence across heterogeneous multi-platform Git checkouts to prevent line-ending conversion churn.

ATOM W7: FILE EXTENSION LANGUAGE ASSOCIATION OVERRIDE File Path: src/main/kotlin/atropos/core/factory/ProjectLanguage.kt Callers and Interfaces: ProjectLanguage enum, called by TreeSitterGrammarBridge, MonacoCodeEditor, and RepoScaffold. Values and Symbols: files.associations map, detect, layoutFor, ProjectLanguageLayout, SupportedLanguages. Imports and Exports: Imports standard collections; exports resolved language syntax descriptors. Implementation Directive: Allow operators to override language detection rules for custom file extensions or specific filenames (such as mapping .config to JSON or .inc to PHP). Ensure the editor loads the appropriate syntax highlighter, linter, and language server automatically. Further Research Directive: Research content-based shebang and heuristic language sniffers that inspect the first 512 bytes of a file when filename extension associations are ambiguous.

ATOM W8: GLOBAL TEXT SEARCH DIRECTORY EXCLUSION File Path: src/main/kotlin/atropos/core/agent/AgentContextCollector.kt Callers and Interfaces: AgentContextCollector class, called by CommandCompleter, FileSearchModal, and GlobalSearchEngine. Values and Symbols: search.exclude map, defaultPackRoots, taskPackPaths, isExcluded, shallowTree. Imports and Exports: Imports java.nio.file.Path, atropos.core.security.ContextPathExclusions; exports filtered search file collections. Implementation Directive: Remove designated directories (like build directories, package caches, minified assets, and binary folders) from global text search queries. Allow operators to override search exclusions per query using dedicated search filter flags. Further Research Directive: Research ripgrep-style memory-mapped parallel search traversals with regex compilation caching for searching million-line codebases under 100 milliseconds.

ATOM W9: SMART CASE-SENSITIVE SEARCH HEURISTIC File Path: src/main/kotlin/atropos/cli/input/FuzzyMatcher.kt Callers and Interfaces: FuzzyMatcher class, called by GlobalSearchEngine, CommandCompleter, and SymbolSearch. Values and Symbols: search.smartCase boolean, matches, distance, normalize, queryHasUppercase. Imports and Exports: Imports java.lang.String; exports case-sensitive or case-insensitive matching verdicts. Implementation Directive: Automatically execute case-insensitive matching when the search query contains only lowercase letters. Switch strictly to case-sensitive matching the moment an uppercase character is typed into the search buffer. Further Research Directive: Research Boyer-Moore-Horspool string search algorithm adaptations that dynamically optimize skip tables based on smart-case character distributions.

ATOM W10: DESTRUCTIVE FILE DELETION CONFIRMATION GATE File Path: apps/web/src/lib/workspace/client.ts Callers and Interfaces: deleteWorkspaceFile function, invoked by WorkspaceFileTree delete action. Values

and Symbols: explorer.confirmDelete boolean, deleteFileAction, showConfirmationDialog, isProtectedPath. Imports and Exports: Imports modal dialog primitives; exports verified file deletion execution calls. Implementation Directive: Intercept file and directory deletion requests in the Explorer pane. Display a prominent confirmation modal warning of permanent data loss, requiring explicit confirmation unless disabled by the operator in settings. Further Research Directive: Research operating system trash bin integration (such as FreeDesktop Trash specification and macOS NSFileManager trashItemAtURL) to support file recovery instead of immediate unrecoverable unlink.

ATOM W11: EXPLORER COMPACT DIRECTORY MERGING File Path: apps/web/src/components/workbench/file-explorer.tsx Callers and Interfaces: FileExplorer component, called by WorkspaceFileTree directory node renderer. Values and Symbols: explorer.compactFolders boolean, compactDirectoryNode, flattenTree, singleChildFolder. Imports and Exports: Imports WorkspaceTreeNode models; exports visual compact folder row nodes. Implementation Directive: Detect nested directory structures that contain only a single sub-folder without sibling files (such as src/main/kotlin). Merge these chained directories into a single compact tree row to reduce vertical scrolling clutter. Further Research Directive: Research tree-flattening graph algorithms that handle rapid directory expansion and partial un-compacting when files are dropped into intermediate sub-folders.

ATOM W12: ACTIVE FILE AUTO-REVEAL IN EXPLORER TREE File Path: apps/web/src/components/workbench/file-explorer.tsx Callers and Interfaces: FileExplorer component, invoked by onActiveTabChange event in EditorTabs. Values and Symbols: explorer.autoReveal boolean, revealFileInTree, expandParents, scrollIntoViewIfNeeded. Imports and Exports: Imports active tab state; exports DOM scroll and directory expansion actions. Implementation Directive: Automatically expand parent directory nodes and scroll the Explorer tree to highlight the currently active editor file whenever the active tab changes. Ensure auto-reveal does not interrupt active user scrolling inside the file pane. Further Research Directive: Research non-blocking DOM scroll-into-view calculations with intersection observers to reveal active items without triggering expensive layout recalculations.

ATOM W13: WORKSPACE WINDOW RESTORATION LIFECYCLE File Path: src/main/kotlin/atropos/core/recovery/StartupContinuationPolicy.kt Callers and Interfaces: StartupContinuationDecider class, called by RuntimeContinuitySupervisor and AppShell. Values and Symbols: window.restoreWindows enum (all, folders, one, none), decide, StartupContinuationReport. Imports and Exports: Imports java.lang.System; exports workspace window restore decisions. Implementation Directive: Control whether previously opened projects, folders, and workspace windows automatically reload upon application startup. Restore all open tabs, terminals, and layout states when enabled, or launch to a clean welcome screen when set to none. Further Research Directive: Research workspace state serialization

schemas using schema versioning to guarantee backward compatibility across engine and IDE upgrades.

ATOM W14: GLOBAL UI ACCESSIBILITY ZOOM LEVEL CONTROLLER File Path: apps/web/src/lib/theme/storage.ts Callers and Interfaces: applyTheme function, called by ThemeProvider and SettingsScreen. Values and Symbols: window.zoomLevel number (scaling factor), applyGlobalZoom, rootFontSize, remScale. Imports and Exports: Imports browser window and document elements; exports applied root CSS zoom variables. Implementation Directive: Provide smooth scaling of all UI text, icons, buttons, and padding across the workspace. Ensure zoom level adjustments maintain proportional high-contrast spacing and never cause text truncation or overlapping layout elements. Further Research Directive: Research CSS zoom property browser compatibility versus root font-size rem scaling and viewport coordinate compensation in Monaco overlays.

ATOM W15: WINDOW TITLE BAR STYLE CUSTOMIZATION File Path: apps/web/src/components/app-shell/app-header.tsx Callers and Interfaces: AppHeader component, rendered at the top of the main application shell. Values and Symbols: window.titleBarStyle enum (custom, native), customTitleBar, windowControlsOverlay, dragRegion. Imports and Exports: Imports React header layout components; exports styled application title bar. Implementation Directive: Support switching between native operating system title bars and integrated custom title bars. In custom mode, house breadcrumbs, global command palette triggers, provider status pills, and window control buttons directly within the top header bar. Further Research Directive: Research Electron and Progressive Web App (PWA) Window Controls Overlay APIs for seamless integration with Windows, macOS, and Linux desktop window frames.

BATCH 9 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (TERMINAL AND DEBUGGING CAPABILITIES)

ATOM T1: DEFAULT TERMINAL SHELL PROFILE DISCOVERY AND SPAWN File Path: src/main/kotlin/atropos/bridge/terminal/TerminalSession.kt Callers and Interfaces: TerminalSession companion object, invoked by EngineHttpServer WebSocket upgrade and CliParityConsoleScreen. Values and Symbols: terminal.integrated.defaultProfile string (bash, zsh, powershell), spawn, getShell, spawnUnixPty, spawnFallback. Imports and Exports: Imports java.lang.ProcessBuilder, java.io.InputStream; exports active TerminalSession process handles. Implementation Directive: Detect installed system shells dynamically on the host environment and launch the configured default profile. Support native POSIX shells (bash, zsh) on Linux/macOS, Termux shells on Android, and PowerShell or cmd on Windows without hardcoded shell paths. Further Research Directive: Research cross-platform shell profile detection via login shell environmental introspection and /etc/shells parsing.

ATOM T2: INTEGRATED TERMINAL SCROLLBACK BUFFER ALLOCA-

TION File Path: apps/web/src/components/workbench/terminal.tsx Callers and Interfaces: TerminalComponent, called by WorkbenchShell bottom panel layout. Values and Symbols: terminal.integrated.scrollback integer (default 1000 to 100000 lines), Terminal options, scrollback, clear. Imports and Exports: Imports Terminal from xterm; exports configured xterm instance with allocated scrollback buffer. Implementation Directive: Configure maximum retained scrollback lines in the terminal emulator. Enforce ring-buffer line recycling when output exceeds the allocated limit, preventing browser tab memory exhaustion during long-running compiler runs. Further Research Directive: Research offscreen circular line buffers and Web Worker memory compaction for retaining massive multi-gigabyte build logs without DOM slowdown.

ATOM T3: TERMINAL CURSOR STYLE AND BLINK CONFIGURATION File Path: apps/web/src/components/workbench/terminal.tsx Callers and Interfaces: TerminalComponent options, called on terminal initialization and settings update. Values and Symbols: terminal.integrated.cursorStyle enum (block, line, underline), cursorBlink boolean, cursorWidth integer. Imports and Exports: Imports xterm terminal theme configuration types; exports styled cursor rendering state. Implementation Directive: Support dynamic switching between block, vertical bar, and underline cursor styles in the integrated terminal. Respect user cursor-blink preferences and adapt cursor dimensions to font metrics automatically. Further Research Directive: Research terminal cursor escape sequence standard compliance (DECSCUSR) for allowing shell programs to dynamically alter cursor styles at runtime.

ATOM T4: MULTI-LINE PASTE ACCIDENTAL EXECUTION GUARD File Path: src/main/kotlin/atropos/cli/input/PastedInputSplitter.kt Callers and Interfaces: PastedInputSplitter object, called by RawKeyReader and TerminalMirrorSocketServer. Values and Symbols: terminal.integrated.enableMultiLinePasteWarning boolean, split, startsCommand, bracketedPasteMode. Imports and Exports: Imports standard Kotlin text utilities; exports sanitized and segmented paste command lists. Implementation Directive: Detect multi-line clipboard paste actions containing newline characters. Prompt the operator for confirmation or collapse content into a preview dialog before execution, preventing accidental immediate submission of unintended commands. Further Research Directive: Research terminal emulator bracketed paste mode protocol extensions across mobile and web sockets to eliminate newline injection vulnerabilities.

ATOM T5: AUTOMATIC COPY-ON-SELECTION TERMINAL INTEGRATION File Path: apps/web/src/components/workbench/terminal.tsx Callers and Interfaces: TerminalComponent text selection listener, invoked by xterm selection change events. Values and Symbols: terminal.integrated.copyOnSelection boolean, onSelectionChange, copyToClipboard, getSelection. Imports and Exports: Imports navigator.clipboard browser APIs; exports copied string payloads to system clipboard. Implementation Directive: Automatically copy highlighted terminal text to the operating system clipboard immediately upon mouse release or touch selection completion. Support

stripping trailing newline artifacts from block-selected terminal text. Further Research Directive: Research browser Permissions API requirements and cross-origin clipboard write capabilities in secure HTTPS versus local HTTP contexts.

ATOM T6: GPU-ACCELERATED HARDWARE TERMINAL RENDERING File Path: apps/web/src/components/workbench/split-terminal-editor.tsx Callers and Interfaces: SplitTerminalEditor component, invoked by SovereignIdeWorkstation layout. Values and Symbols: terminal.integrated.gpuAcceleration enum (auto, on, off), WebglAddon, CanvasAddon, FitAddon. Imports and Exports: Imports xterm addon packages; exports hardware-accelerated terminal canvas renderer. Implementation Directive: Offload terminal text rendering and glyph drawing to the GPU using WebGL or Canvas2D acceleration. Automatically fall back to software DOM rendering if WebGL context creation fails or graphics hardware is unavailable. Further Research Directive: Research WebGL texture atlas packing for monospace font glyphs to maximize text rendering throughput under extreme terminal throughput bursts.

ATOM T7: OMNIPRESENT BREAKPOINT PLACEMENT SUBSYSTEM File Path: src/main/kotlin/atropos/core/parser/TreeSitterGrammarBridge.kt Callers and Interfaces: TreeSitterGrammarBridge class, called by DebuggerBreakpointService and MonacoCodeEditor. Values and Symbols: debug.allowBreakpointsEverywhere boolean, parseTree, walkTree, validateBreakpointPosition, lineLocation. Imports and Exports: Imports atropos.core.parser.KotlinParseTree; exports validated breakpoint line coordinates. Implementation Directive: Allow operators to place breakpoint markers on any line of code across any file type. Validate breakpoint positions against parsed AST syntax structures to snap pause points to the nearest executable expression. Further Research Directive: Research Debug Adapter Protocol (DAP) source mapping and bytecode line-number table alignment across mixed-language projects.

ATOM T8: INLINE VARIABLE VALUE EDITOR OVERLAYS File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor decoration provider, called by debug session step events. Values and Symbols: debug.inlineValues boolean, setViewDecorations, variableStateMap, inlineValueFormat. Imports and Exports: Imports editor decoration types; exports visual inline variable annotation widgets. Implementation Directive: Render live variable values, object properties, and evaluation results directly adjacent to variable references within editor code lines during active debug pause states. Update inline annotations instantly upon execution stepping. Further Research Directive: Research Monaco Editor deltaDecoration APIs for high-performance DOM text injections that avoid line re-wrapping or cursor layout instability.

ATOM T9: DEBUG CONSOLE AUTOMATIC FOCUS ORCHESTRATION File Path: apps/web/src/components/workbench/log-panel.tsx Callers and In-

terfaces: LogPanel component, called by debug session lifecycle manager. Values and Symbols: debug.internalConsoleOptions enum (neverOpen, openOnFirstSessionStart, openOnSessionStart), activateTab. Imports and Exports: Imports workbench panel focus controllers; exports active tab switching actions. Implementation Directive: Automatically bring the Debug Console panel into active focus when a debug session launches or hits its first breakpoint, according to user configuration. Prevent unwanted panel focus stealing during ordinary test runs. Further Research Directive: Research non-intrusive notification badge systems that alert developers to debug outputs without forcibly stealing keyboard focus from active code editing.

ATOM T10: GLOBAL STATUS BAR DEBUG SESSION CONTROLS File Path: src/main/kotlin/atropos/cli/ui/BottomStatusBar.kt Callers and Interfaces: BottomStatusBar class, called by AnsiTerminalEngine and Web HeaderRenderer. Values and Symbols: debug.showInStatusBar enum (never, always, onFirstSessionStart), activeDebugSession, pauseAction, stopAction. Imports and Exports: Imports atropos.cli.ui.design.Role; exports status bar debug control widgets. Implementation Directive: Surface compact execution controls (play, pause, step over, step into, restart, terminate) directly within the global bottom status bar when a debug session is active, maintaining control access when panels are closed. Further Research Directive: Research compact SVG icon button rendering in narrow mobile status bars with accessible high-contrast active states.

ATOM T11: EXECUTION CONTROL FLOATING TOOLBAR REPOSITIONING File Path: apps/web/src/components/visual/command-dock.tsx Callers and Interfaces: CommandDock component, invoked by SovereignIdeWorkstation editor viewport. Values and Symbols: debug.toolBarLocation enum (floating, docked, hidden), dockPosition, isDragging, snapToEdge. Imports and Exports: Imports React drag-and-drop primitives; exports repositionable floating debug control widget. Implementation Directive: Provide a floating, draggable debug toolbar widget overlaying the active editor. Allow developers to dock the widget to the editor top header, move it freely, or pin it into the debug activity bar pane. Further Research Directive: Research drag-boundary containment and touch gesture velocity snapping algorithms for floating utility docks across mobile and desktop browser viewports.

ATOM T12: DEBUG CONSOLE OUTPUT WORD WRAPPING File Path: apps/web/src/components/workbench/log-panel.tsx Callers and Interfaces: LogPanel component, called during debug output line appending passes. Values and Symbols: debug.console.wordWrap boolean, consoleLineWrap, preformattedWrapStyle. Imports and Exports: Imports CSS styling primitives; exports wrapped console log entry rows. Implementation Directive: Enable automatic soft word-wrapping within the Debug Console output area. Prevent horizontal scrolling when evaluating and printing deeply nested object structures or lengthy JSON stack trace payloads. Further Research Directive: Research virtualized ANSI-colored text wrapping algorithms that calculate line heights

dynamically for variable-width log items.

ATOM T13: AUTOMATED LAZY VARIABLE OBJECT EXPANSION File Path: apps/web/src/components/workbench/log-panel.tsx Callers and Interfaces: LogPanel variable tree view, called by debug variable inspection handlers. Values and Symbols: debug.autoExpandLazyVariables boolean, evaluateLazyProperty, expandObjectTree, maxDepth. Imports and Exports: Imports variable inspection interfaces; exports evaluated object graph structures. Implementation Directive: Automatically trigger evaluation and expansion of lazy properties, getters, and nested object fields in the debug variables tree view. Bound recursive evaluation depth to prevent circular reference stack overflows. Further Research Directive: Research asynchronous object graph pagination protocols in DAP to stream large collection elements on demand without freezing the debug adapter.

ATOM T14: EXPERIMENTAL JAVASCRIPT AND NODE DEBUGGER ADAPTER File Path: src/main/kotlin/atropos/core/toolchain/drivers/TypescriptNodeDriver.kt Callers and Interfaces: TypescriptNodeDriver class, called by LanguageToolchainDriver and debug launch tasks. Values and Symbols: debug.javascript.usePreview boolean, launchNodeInspector, debugPort, cdpSession. Imports and Exports: Imports atropos.core.toolchain.ToolchainResult; exports Node.js debug process launch configs. Implementation Directive: Support debugging Node.js, TypeScript, and browser JavaScript runtimes using Chrome DevTools Protocol (CDP) socket connections. Support setting breakpoints, inspecting call stacks, and evaluating live console expressions. Further Research Directive: Research V8 inspector protocol WebSocket multiplexing for simultaneous client-side and server-side JavaScript full-stack debugging.

ATOM T15: EXPLORER VIEW AUTO-RESTORATION ON DEBUG TERMINATION File Path: apps/web/src/components/app-shell/workbench-main.tsx Callers and Interfaces: WorkbenchMain component, called on debug session completion listener. Values and Symbols: debug.openExplorerOnEnd boolean, onDebugSessionEnd, switchActivityView, previousViewStore. Imports and Exports: Imports view container state hooks; exports active sidebar panel transition actions. Implementation Directive: Automatically switch the active sidebar panel back to the File Explorer pane when a debug session finishes execution. Restore previous editor focus and layout states seamlessly without requiring manual sidebar clicks. Further Research Directive: Research user intent state machines that remember pre-debug view states to restore the developer exact working layout automatically.

BATCH 10 OF 11: ATROPOS SOURCE DOC 10 DECOMPOSITION (CODESPACES, CLOUD INTEGRATIONS & VERSION CONTROL)

ATOM CC1: CLOUD WORKSPACE INACTIVITY SUSPENSION MONITOR File Path: src/main/kotlin/atropos/core/agent/AgentDaemonModels.kt Callers and Interfaces: AgentDaemonRecord data class, invoked by AgentDae-

monService and ObjectLeaseStore. Values and Symbols: codespaces.idleTimeout integer (minutes before suspend), isStale, heartbeat, pollSeconds. Imports and Exports: Imports java.time.Instant; exports daemon lifecycle and suspension telemetry. Implementation Directive: Monitor operator interactivity and running task queues. Automatically trigger container suspension or resource scaling when inactivity exceeds the configured threshold, preserving workspace disk state while eliminating idle compute costs. Further Research Directive: Research zero-downtime Linux container cgroup freezer APIs to freeze and resume containerized workloads without terminating active network sockets.

ATOM CC2: REMOTE COMPUTE HARDWARE FLAVOR SELECTOR File Path: src/main/kotlin/atropos/core/platform/PlatformModels.kt Callers and Interfaces: PlatformDescriptor data class, invoked by HardwareProfileAdapter and DeploymentService. Values and Symbols: codespaces.machineType string (such as standard-4core-16gb, gpu-a10g), cpuCores, totalMemoryBytes. Imports and Exports: Imports standard platform descriptors; exports target virtual machine specifications. Implementation Directive: Define the hardware resource tier (CPU core count, RAM allocation, GPU acceleration) for remote and cloud instances. Provide seamless scaling requests between lightweight mobile companion nodes and heavy compilation servers. Further Research Directive: Research dynamic memory ballooning in KVM and Firecracker microVMs for elastic RAM allocation based on active compiler demands.

ATOM CC3: CLOUD WORKSPACE STORAGE RETENTION AND PRUNING File Path: src/main/kotlin/atropos/core/storage/RetentionPolicy.kt Callers and Interfaces: RetentionPolicy class, called by BlobStoreGc, EvidenceBundleGc, and StorageSupervisor. Values and Symbols: codespaces.retentionPeriod integer (days), ruleFor, tierFor, StorageRetentionRule, GcWatermark. Imports and Exports: Imports java.time.Duration; exports storage garbage collection policies. Implementation Directive: Enforce retention deadlines on stopped or abandoned cloud workspaces. Archive persistent workspace volumes to cold storage and purge unreferenced environments after the retention window expires to enforce storage ceilings. Further Research Directive: Research delta-snapshot deduplication algorithms over ZFS or btrfs block volumes for storing long-term dormant workspace archives at minimal cost.

ATOM CC4: SECURE CLOUD CONTAINER SECRETS INJECTION File Path: src/main/kotlin/atropos/core/security/TokenIsolationVault.kt Callers and Interfaces: TokenIsolationVault class, called by CredentialBootstrap and AppProjectMutationGate. Values and Symbols: codespaces.secretsManagement boolean, readSecretForSink, SecretSinkKind, VaultReadResult. Imports and Exports: Imports java.nio.file.Path; exports decrypted environment variable key-value pairs. Implementation Directive: Inject user API keys and environment variables securely into remote container environments at boot time. Ensure credentials reside purely in memory (tmpfs) and are never baked into container images or committed into Git history. Further Research Directive: Research Linux kernel keyrings (keyctl) and systemd-creds encryption for

delivering encrypted environment variables directly to container init processes.

ATOM CC5: AUTOMATIC CONTAINER PORT FORWARDING ENGINE File Path: src/main/kotlin/atropos/bridge/LocalHttpServer.kt Callers and Interfaces: LocalHttpServer class, invoked by EngineHttpServer and LivePreviewStudio. Values and Symbols: codespaces.portForwarding boolean, forwardPort, publicUrl, isPrivate, bindAddress. Imports and Exports: Imports java.net.ServerSocket; exports mapped local and public port tunnel addresses. Implementation Directive: Detect active TCP listeners spawned by user web servers or microservices inside the workspace container. Automatically map internal ports to secure, authenticated local loopback or public URLs with live status indicators. Further Research Directive: Research zero-overhead HTTP reverse-proxy multiplexing using SNI routing to tunnel multiple internal ports over a single TLS connection.

ATOM CC6: AUTOMATED USER DOTFILES REPOSITORY CLONER File Path: src/main/kotlin/atropos/core/worktree/BoundedGitWorktreeCommandRunner.kt Callers and Interfaces: BoundedGitWorktreeCommandRunner class, called during workspace environment provisioning. Values and Symbols: codespaces.dotfilesRepository string (repo URI), run, safeCommitMessage, GitWorktreeOperation. Imports and Exports: Imports java.nio.file.Path; exports Git clone and setup script execution results. Implementation Directive: Automatically clone and apply a developer personal dotfiles repository (.bashrc, .zshrc, .gitconfig) upon container initialization. Execute bootstrap install scripts within sandboxed limits to personalize shell environments. Further Research Directive: Research isolated home-directory overlay filesystems to apply user shell customizations without polluting project repository files.

ATOM CC7: WORKSPACE LIFECYCLE HOOKS EXECUTION RUNNER File Path: src/main/kotlin/atropos/core/policy/BoundedProcessRunner.kt Callers and Interfaces: BoundedProcessRunner class, invoked by AppFactoryRouter and AutonomousOrchestrator. Values and Symbols: codespaces.lifecycleScripts map (postCreate, postStart, updateContent), runProcess, BoundedProcessResult. Imports and Exports: Imports java.lang.Process; exports exit codes and output logs from lifecycle hooks. Implementation Directive: Execute declared lifecycle scripts automatically during workspace lifecycle phases (after repository clone, upon container start, or upon dependency changes). Stream logs directly to the integrated output console. Further Research Directive: Research non-blocking process supervision trees with strict timeout ceilings to prevent hung post-create scripts from freezing container startup.

ATOM CC8: REPOSITORY DEVCONTAINER SPECIFICATION PARSER File Path: src/main/kotlin/atropos/core/factory/AppProjectSpec.kt Callers and Interfaces: AppProjectSpecParser class, invoked by WorkspaceIntelligence and ToolchainExecutionDriver. Values and Symbols: codespaces.devcontainerJsonConfiguration boolean, parse, buildSpecs, containerImage, featuresMap. Imports and Exports: Imports JSON parsing utilities; exports structured DevContainer

environment models. Implementation Directive: Parse repository .devcontainer/devcontainer.json manifests to configure container base images, pre-installed toolchains, IDE extensions, and runtime forwardPorts automatically. Further Research Directive: Research Open Container Initiative (OCI) image layer caching optimizations to accelerate devcontainer rebuilds from pre-warmed image snapshots.

ATOM CC9: INTEGRATED BROWSER PREVIEW WORKSPACE TAB File Path: apps/web/src/components/studio/live-preview-studio.tsx Callers and Interfaces: LivePreviewStudio component, rendered as an editor tab or split pane view. Values and Symbols: codespaces.browserPreview boolean, DeviceFrameContainer, captureLiveUrl, hotReload. Imports and Exports: Imports browser actuator and visual comparison drivers; exports embedded preview viewport. Implementation Directive: Render a lightweight, sandboxed browser viewport directly inside the IDE to display running local web applications. Provide instant hot-reload refreshes, responsive viewport resizing, and console log capture. Further Research Directive: Research browser iframe sandbox restrictions and Content Security Policy (CSP) headers to isolate untrusted user-rendered scripts from the IDE parent frame.

ATOM CC10: GITHUB PULL REQUEST CREATION ROUTER File Path: src/main/kotlin/atropos/core/github/GitHubApiClient.kt Callers and Interfaces: GitHubApiClient class, called by GitHubBinding and SentryRepairCoordinator. Values and Symbols: github.pullRequests.createOnGithub boolean, createPullRequest, GitHubWriteAuthorization, prUrl. Imports and Exports: Imports java.net.http.HttpClient; exports created Pull Request metadata and web links. Implementation Directive: Allow operators to choose between creating Pull Requests natively inside the IDE or redirecting to the official GitHub web interface with pre-filled branch, title, and body parameters. Further Research Directive: Research GitHub GraphQL API v4 mutations for atomic multi-commit Pull Request creation with automated reviewer assignment.

ATOM VC1: PERIODIC BACKGROUND GIT AUTOFETCH File Path: src/main/kotlin/atropos/core/worktree/BoundedGitWorktreeCommandRunner.kt Callers and Interfaces: BoundedGitWorktreeCommandRunner class, invoked by periodic background daemon timer. Values and Symbols: git.autofetch enum (all, current, off), autofetchIntervalSeconds, GitWorktreeOperation.FETCH. Imports and Exports: Imports atropos.core.policy.BoundedProcessRunner; exports remote branch tracking state. Implementation Directive: Periodically query remote Git repositories in the background to fetch updated commit references. Update incoming and outgoing commit counters in the status bar without locking the active working index. Further Research Directive: Research Git fetch --prune optimizations using lightweight packfile negotiation protocols to minimize background network bandwidth.

ATOM VC2: COMBINED PULL AND PUSH CONFIRMATION PROMPT File Path: apps/web/src/components/ide/sovereign-ide-workstation.tsx Callers

and Interfaces: SovereignIdeWorkstation component, invoked by Git synchronization action button. Values and Symbols: git.confirmSync boolean, showSyncConfirmation, syncButtonState, hasRemoteDivergence. Imports and Exports: Imports Git client hooks; exports interactive sync confirmation dialogs. Implementation Directive: Prompt the operator for explicit confirmation before executing combined pull and push operations. Warn developers if the local branch has diverged significantly from remote tracking branches. Further Research Directive: Research non-blocking notification toasts with inline undo actions for non-destructive Git synchronization workflows.

ATOM VC3: SMART AUTOMATIC STAGING FOR UNSTAGED COMMITS File Path: src/main/kotlin/atropos/cli/shell/ShellCommandRunner.kt Callers and Interfaces: ShellCommandRunner class, called by GitMutationCommandParser and AgentPatchStore. Values and Symbols: git.enableSmartCommit boolean, runGitMutation, gitStatus, stageAllModifiedFiles. Imports and Exports: Imports atropos.core.policy.ActionProposal; exports Git commit execution receipts. Implementation Directive: Automatically stage all tracked modified files if a commit command is executed when the staging area is empty. Warn the operator if untracked files exist and require explicit approval before staging new files. Further Research Directive: Research atomic Git index locking mechanisms to prevent concurrent file modifications from leaking into in-flight smart commit operations.

ATOM VC4: POST-COMMIT AUTOMATED WORKFLOW ACTIONS File Path: src/main/kotlin/atropos/core/artifact/ArtifactPipeline.kt Callers and Interfaces: ArtifactPipeline class, called by FinalConvergenceReleaseGate and GitMutationCommandParser. Values and Symbols: git.postCommitCommand enum (none, push, sync), finalizeCommit, CommitCandidate. Imports and Exports: Imports atropos.core.github.GitHubBinding; exports post-commit workflow triggers. Implementation Directive: Execute configured secondary actions automatically following a successful local Git commit, such as triggering an immediate push to remote tracking branches or launching automated CI validation passes. Further Research Directive: Research transactional commit-and-push pipelines that automatically roll back local commits if remote push rejected due to protected branch policies.

ATOM VC5: DEDICATED THREE-WAY MERGE CONFLICT EDITOR File Path: apps/web/src/components/ide/unified-diff-editor.tsx Callers and Interfaces: UnifiedDiffEditor component, invoked when Git reports unmerged conflict states. Values and Symbols: git.mergeEditor boolean, UnifiedDiffEditorProps, originalContent, modifiedContent, resolveConflict. Imports and Exports: Imports diff models; exports resolved merged text buffer strings. Implementation Directive: Present a dedicated three-way merge editor interface displaying base, current, and incoming change hunks side by side. Provide one-click buttons to accept current, incoming, or combined changes cleanly. Further Research Directive: Research AST-aware semantic merge conflict resolution algorithms that automatically resolve non-overlapping syntactic changes in code

declarations.

ATOM VC6: DESTRUCTIVE REBASE WARNING SUPPRESSION File Path: src/main/kotlin/atropos/cli/GitMutationCommand.kt Callers and Interfaces: GitMutationCommandParser object, called by InteractiveCliRunner and ShellCommandHandler. Values and Symbols: git.ignoreRebaseWarning boolean, parse, parseRollback, safeRelativePath. Imports and Exports: Imports standard collections; exports parsed Git mutation commands. Implementation Directive: Provide a configuration toggle to silence warning confirmations when performing interactive rebases or force-with-lease updates, intended for experienced developers while defaulting to safe confirmation warnings. Further Research Directive: Research reflog safety anchor creation prior to rebase invocations, enabling instantaneous one-click rollback if rebase operations fail.

ATOM VC7: RECURSIVE PARENT DIRECTORY GIT ROOT DISCOVERY File Path: src/main/kotlin/atropos/core/AtroposRepoRootLocator.kt Callers and Interfaces: AtroposRepoRootLocator object, called by TargetWorkspace-Locator, AgentContextCollector, and CLI main. Values and Symbols: git.openRepositoryInParentFolders enum (always, never, prompt), resolve, isAtroposRoot, walkUp. Imports and Exports: Imports java.nio.file.Path, java.nio.file.Files; exports discovered repository root path. Implementation Directive: Walk upwards through parent directories from the current working folder to discover enclosing Git repositories and .atropos configuration markers. Stop traversing at filesystem mounts or user home roots. Further Research Directive: Research boundary filesystem detection across POSIX mountpoints using statfs system calls to avoid unbounded directory traversal over network mounts.

ATOM VC8: MANDATORY TRACKING FETCH BEFORE PULL MERGE File Path: src/main/kotlin/atropos/core/worktree/BoundedGitWorktreeCommandRunner.kt Callers and Interfaces: BoundedGitWorktreeCommandRunner class, invoked during Git pull workflows. Values and Symbols: git.fetchOnPull boolean, run, GitWorktreeOperation.PULL, fetchRemoteTracking. Imports and Exports: Imports process runner interfaces; exports synchronized tracking branch refs. Implementation Directive: Ensure that local Git tracking branches are explicitly updated via git fetch prior to executing pull and merge commands. Prevent local merge commits against stale tracking branches. Further Research Directive: Research fast-forward only pull configurations (pull.ff = only) to enforce linear Git commit histories across autonomous agent workspaces.

ATOM VC9: MAIN BRANCH ACCIDENTAL PUSH PROTECTION GUARD File Path: src/main/kotlin/atropos/core/security/ProtectedPathPolicy.kt Callers and Interfaces: ProtectedPathPolicy class, called by GitHubBinding, SelfHostPromotionService, and ShellCommandHandler. Values and Symbols: git.branchProtection boolean, checkCommand, protectedBranches list (main, master, release), ProtectedPathResult. Imports and Exports: Imports java.nio.file.Path; exports branch write authorization verdicts. Implementation Directive: Intercept outgoing Git push commands targeting protected branches

(such as main or master). Block direct pushes from autonomous agents or accidental user actions, requiring feature branch creation and pull requests. Further Research Directive: Research Git pre-push hook integration with signed cryptographic branch policies to enforce pull request validation at the protocol level.

ATOM VC10: STATUS BAR BACKGROUND GIT PROGRESS MONITOR File Path: src/main/kotlin/atropos/cli/ui/BottomStatusBar.kt Callers and Interfaces: BottomStatusBar class, called by TerminalRenderingFacade and AnsiTerminalEngine. Values and Symbols: git.showProgress boolean, branch, render, dirtyMarker, incomingCount, outgoingCount. Imports and Exports: Imports atropos.cli.ui.design.Role; exports status bar Git progress widgets. Implementation Directive: Render background Git operation indicators (fetching, indexing, rebasing, committing) directly in the bottom status bar with spinning activity glyphs and sync counters, keeping developers informed without modal popups. Further Research Directive: Research non-blocking polling loops that read Git index and packed-refs metadata without executing full git status subprocesses.

BATCH 11 OF 12: ATROPOS SOURCE DOC 10 DECOMPOSITION (HIGH-VALUE EXTENSIONS & INTEGRATIONS)

ATOM EX1: GITHUB PULL REQUESTS AND ISSUES NATIVE WORKSPACE PANEL File Path: src/main/kotlin/atropos/core/github/GitHubRemoteMcpTool.kt Callers and Interfaces: GitHubRemoteMcpTool class, invoked by GenericMcpHostManager, AppSidebar, and GitHubBinding. Values and Symbols: listOpenPullRequests, createPullRequest, postComment, PullRequestRecord data class, tokenProvider. Imports and Exports: Imports org.json.JSONObject, java.net.HttpURLConnection; exports native repository collaboration models. Implementation Directive: Provide a dedicated sidebar panel to inspect, browse, review, and comment on open repository pull requests and issues without switching to external web browsers. Support inline review comment threading and one-click branch checkout. Further Research Directive: Research GitHub REST API conditional ETag caching to poll pull request review threads without consuming rate limit quotas.

ATOM EX2: GITLENS INLINE REPOSITORY BLAME AND HEATMAP EXPLORER File Path: src/main/kotlin/atropos/core/github/GitHubBinding.kt Callers and Interfaces: GitHubBinding class, invoked by MonacoCodeEditor inline annotations and HistoryNavigationRenderer. Values and Symbols: fileBlame, revision, commitSha, authorName, authorTimestamp, blameLinesMap, heatMapDistribution. Imports and Exports: Imports atropos.core.worktree.BoundedGitWorktreeCommandRunner; exports line-level author and commit provenance. Implementation Directive: Render subtle, non-intrusive inline annotations at the end of the active code line showing the author, commit age, and commit message. Expose on-hover heatmaps and graphical commit graphs detailing repository modification frequency. Further Research Directive: Research git blame incremental streaming parsing over

memory buffers to display blame annotations instantly on files exceeding ten thousand lines.

ATOM EX3: PRETTIER OPINIONATED MULTI-LANGUAGE FORMATTER INTEGRATION File Path: src/main/kotlin/atropos/core/factory/StagingFileWriter.kt Callers and Interfaces: StagingFileWriter object, called by MonacoCodeEditor onSave pipeline and CodeBlockExtractor. Values and Symbols: normalize, writeSource, runPrettierFormat, printWidth, tabWidth, singleQuote, trailingComma. Imports and Exports: Imports java.nio.file.Path, java.nio.file.StandardOpenOption; exports formatted source code buffers. Implementation Directive: Integrate an opinionated code formatter supporting JavaScript, TypeScript, CSS, JSON, Markdown, and YAML. Enforce consistent stylistic rules across all project files on save, eliminating style debates and formatting diff churn. Further Research Directive: Research standalone WebAssembly-compiled Prettier bundles for instantaneous in-browser formatting without invoking Node.js subprocesses.

ATOM EX4: ESLINT STATIC ANALYSIS AND ERROR LINTER File Path: src/main/kotlin/atropos/core/telemetry/StaticAnalysisAndLinterDefectHarvester.kt Callers and Interfaces: StaticAnalysisAndLinterDefectHarvester class, invoked by TypescriptNodeDriver and VerifiedCompletionGate. Values and Symbols: calculateDefectPenalty, recordLinterDefect, linterRuleId, severityLevel, complexityMetrics. Imports and Exports: Imports atropos.core.autonomy.RewardPenaltyStore; exports recorded syntax and code-smell defects. Implementation Directive: Run real-time static code analysis over JavaScript and TypeScript source files. Highlight anti-patterns, potential memory leaks, type coercions, and unused variables with colored squiggly underlines and provide automatic quick-fix actions. Further Research Directive: Research worker-thread AST linting architectures that compute linter diagnostics incrementally while code is typed without dropping editor frame rates.

ATOM EX5: PYLANCE ADVANCED PYTHON TYPE-CHECKING ENGINE File Path: src/main/kotlin/atropos/core/toolchain/drivers/PythonFastApiDriver.kt Callers and Interfaces: PythonFastApiDriver class, called by LanguageToolchainDriver and DeterministicHeuristicRepairEngine. Values and Symbols: parseMypyResult, parseRuffResult, parseMypyOutput, resolvePython, ToolchainDiagnostic. Imports and Exports: Imports atropos.core.policy.BoundedProcessResult; exports typed Python diagnostics and symbol maps. Implementation Directive: Deliver deeply integrated static type checking, parameter hints, auto-imports, and navigation for Python projects. Validate function signatures, return types, and class attributes against declared typing definitions. Further Research Directive: Research Pyright language server JSON-RPC protocol initialization over standard IO pipes for embedded high-performance Python analysis.

ATOM EX6: DOCKER EXPLORER CONTAINER AND VOLUME MANAGEMENT File Path: src/main/kotlin/atropos/core/factory/AppDeploymentService.kt

Callers and Interfaces: DeploymentService class, invoked by AutonomousDeploymentSeam and CloudDeploymentAdapter. Values and Symbols: produceOciImage, packageLocalDist, imageName, containerRegistry, listContainers, inspectVolume. Imports and Exports: Imports java.nio.file.Path, java.util.concurrent.ConcurrentHashMap; exports OCI image build digests. Implementation Directive: Provide an integrated panel to inspect, start, stop, and prune local Docker containers, images, volumes, and networks. Enable one-click inspection of container output logs and interactive terminal execution inside running containers. Further Research Directive: Research rootless Docker and Podman socket integration on Linux and Termux environments to manage containerized development without root daemon privileges.

ATOM EX7: REMOTE SSH PHYSICAL SERVER FILE AND TERMINAL ACCESS File Path: src/main/kotlin/atropos/core/toolchain/ChrootUnshareJail.kt Callers and Interfaces: ChrootUnshareJail class, invoked by PlatformWire and TerminalSession. Values and Symbols: wrapCommand, stripSecretsFromEnvironment, sshHost, sshPort, sshIdentityKey, remoteWorkspacePath. Imports and Exports: Imports java.nio.file.Path; exports sandboxed remote process execution wrappers. Implementation Directive: Enable opening remote folders and executing shell commands on external physical servers or virtual machines over secure SSH tunnels. Synchronize editor file buffers, language servers, and terminal sessions transparently across the network. Further Research Directive: Research SSH connection multiplexing (ControlMaster) and persistent connection keeping to prevent remote editing lag over high-latency networks.

ATOM EX8: REMOTE WSL NATIVE LINUX SUBSYSTEM INTERACTION File Path: src/main/kotlin/atropos/core/platform/PlatformAdapter.kt Callers and Interfaces: PlatformAdapterRegistry object, called by ToolchainExecutionDriver on Windows host systems. Values and Symbols: PlatformAdapter interface, wslDistribution, resolveWslPath, convertWindowsPath, isAvailable. Imports and Exports: Imports atropos.core.platform.PlatformAbstraction; exports Windows-to-Linux bridge adapters. Implementation Directive: Provide seamless interoperability with the Windows Subsystem for Linux (WSL). Execute compilers, Git operations, and terminal sessions inside the native Linux environment while editing files smoothly on the Windows host surface. Further Research Directive: Research 9P filesystem protocol throughput bottlenecks when editing Windows-mounted files inside WSL2 and implement native Linux worktree caching.

ATOM EX9: REMOTE CONTAINERS REPRODUCIBLE ISOLATION MOUNT File Path: src/main/kotlin/atropos/core/toolchain/drivers/DatabaseContainerMockSandboxDriver.k Callers and Interfaces: DatabaseContainerMockSandboxDriver class, invoked by WaveBatchExecutionCoordinator. Values and Symbols: openEphemeralConnection, rejectLiveCredentials, validateWorktree, containerMountPath, isolationLevel. Imports and Exports: Imports java.sql.Connection, atropos.core.toolchain.ToolchainResult; exports sandboxed test database handles. Implementation Directive: Mount the entire development workspace inside

an isolated, disposable container defined by repository manifests. Guarantee reproducible compiler environments, system libraries, and database services isolated from the host machine. Further Research Directive: Research overlay2 filesystem mounts and bind-mount caching options (:cached, :delegated) to achieve native compilation speeds inside mounted containers.

ATOM EX10: LIVE SHARE REAL-TIME COLLABORATIVE EDITING BUS File Path: apps/web/src/lib/bridge/binary-ipc-socket.ts Callers and Interfaces: BinaryIpcSocket class, invoked by SovereignIdeWorkstation and CollaborativeSessionManager. Values and Symbols: BinaryIpcSocket, sendInput, onFrame, SocketState, FrameListener, sharedCursorMap. Imports and Exports: Imports browser WebSocket primitives; exports real-time collaborative state synchronization. Implementation Directive: Support multi-user collaborative code editing, shared terminal viewing, and co-debugging sessions over secure peer connections. Render remote collaborator cursor locations and selections in distinct user colors. Further Research Directive: Research Conflict-Free Replicated Data Types (CRDTs such as Yjs or Automerge) for zero-latency peer-to-peer code synchronization with offline conflict resolution.

ATOM EX11: PLAIN-TEXT REST CLIENT API QUERY EXECUTOR File Path: src/main/kotlin/atropos/core/ProviderHttpClient.kt Callers and Interfaces: ProviderHttpClient class, called by McpHostManager and DeveloperToolsContainer. Values and Symbols: postJson, getJson, requestBounded, BoundedResponse, extractOpenAiChatContent. Imports and Exports: Imports java.net.http.HttpClient, atropos.core.security.RedactionFilter; exports HTTP query execution results. Implementation Directive: Allow operators to author and execute raw HTTP and REST queries directly from .http or .rest plaintext files inside the workspace. Display headers, response payloads, status codes, and execution timings in a side-by-side response panel. Further Research Directive: Research multipart form-data streaming and GraphQL query syntax parsing in lightweight zero-dependency Kotlin HTTP clients.

ATOM EX12: THUNDER CLIENT POSTMAN-STYLE GUI ROUTE TESTER File Path: src/main/kotlin/atropos/core/endpoint/OperationRegistry.kt Callers and Interfaces: OperationRegistry interface, implemented by StaticOperationRegistry and ProviderTruthOperationRegistry. Values and Symbols: getAll, getById, getByKind, OperationEndpoint, OperationManifest, EndpointKind. Imports and Exports: Imports atropos.core.endpoint.EndpointKind; exports registered API endpoint declarations. Implementation Directive: Provide a clean visual API testing interface embedded within the IDE. Enable developers to craft requests, configure query parameters and authentication headers, test JSON payloads, and inspect formatted response bodies without external tools. Further Research Directive: Research automated OpenAPI and Swagger specification generation from recorded visual API test collections.

ATOM EX13: SUPABASE POSTGRESQL DIRECT SCHEMA AND QUERY INTEGRATION File Path: apps/web/src/lib/auth/server.ts Callers and Inter-

faces: createSupabaseServerClient function, called by DatabaseContainerMockSandboxDriver and AdminPortal. Values and Symbols: createSupabaseServerClient, supabaseUrl, supabaseAnonKey, executeSqlRpc, inspectSchemaTables. Imports and Exports: Imports createServerClient from supabase/ssr; exports authenticated database client instances. Implementation Directive: Connect directly to the project Supabase PostgreSQL instance. Allow developers to inspect database tables, browse schemas, inspect Row-Level Security policies, and execute raw SQL queries directly from an IDE management tab. Further Research Directive: Research PostgreSQL schema diffing tools in TypeScript to generate atomic database migration scripts from visual schema changes.

ATOM EX14: PRISMA ORM SCHEMA HIGHLIGHTER AND CODE GENERATOR File Path: src/main/kotlin/atropos/core/contract/SqlDdlEmitter.kt Callers and Interfaces: SqlDdlEmitter object, invoked by ContractLatticeSynthesizer and AppDatabaseSecurityPlanner. Values and Symbols: emit, formatDdl, parsePrismaSchema, generateDataModels, validateModelRelations. Imports and Exports: Imports standard collections; exports normalized SQL DDL scripts and data access models. Implementation Directive: Provide syntax highlighting, relationship linting, and autocomplete for schema.prisma files. Validate model relationships, foreign keys, and unique indices, offering automated migration generation and client library compilation. Further Research Directive: Research language server protocol extensions tailored to custom ORM domain-specific languages for instant validation of database schema constraints.

ATOM EX15: TAILWIND CSS INTELLISENSE AND CLASS AUTOCOMPLETE File Path: apps/web/src/components/studio/theme-playground.tsx Callers and Interfaces: ThemePlayground component, invoked by MonacoCodeEditor CSS class provider. Values and Symbols: tailwindClassSuggestions, cssUtilityColorPreview, autocompleteClassPrefix, resolveThemeTokens. Imports and Exports: Imports design token theme palettes; exports Tailwind utility class completion lists. Implementation Directive: Provide intelligent code completion, hover previews, and syntax linting for Tailwind CSS utility classes in HTML, JSX, and TSX files. Render inline color swatches next to color classes and warn about conflicting utilities. Further Research Directive: Research Tailwind CSS PostCSS AST scanning in Web Workers to generate project-specific custom utility classes dynamically from tailwind.config.js files.

BATCH 12 OF 12: ATROPOS SOURCE DOC 10 DECOMPOSITION (MCP & AI CAPABILITIES)

ATOM AI1: GITHUB COPILOT CHAT WORKSPACE MCP INTEGRATION File Path: src/main/kotlin/atropos/core/integration/GenericMcpHostManager.kt Callers and Interfaces: GenericMcpHostManager class, called by SovereignConversationWorkstation and McpCommandHandler. Values and Symbols: GenericMcpServerConfig, GenericMcpToolDescriptor, callTool, workspaceChatContext, copilotPromptFormat. Imports and Exports: Imports org.json.JSONObject, java.util.concurrent.ConcurrentHashMap; exports active Copilot MCP bridge handles. Implementation Directive: Integrate con-

versational AI assistance deeply grounded in active workspace context. Enable the agent to index project symbols, inspect active open editor files, answer architectural queries, and suggest in-place refactorings. Further Research Directive: Research retrieval-augmented generation (RAG) chunking strategies optimized for code context token windows up to 128k tokens.

ATOM AI2: CLAUDE CODE AUTONOMOUS AGENT MCP TOOLCHAIN File Path: src/main/kotlin/atropos/core/provider/ProviderImplementations.kt Callers and Interfaces: AnthropicProvider class, invoked by InboundToolBridge and AutonomousOrchestrator. Values and Symbols: AnthropicProvider, complete, extractAnthropicText, claudeToolUsePayload, executeCommandTool. Imports and Exports: Imports atropos.core.security.RedactionFilter; exports Claude model inference and autonomous tool call dispatches. Implementation Directive: Support Claude agent tool calling to autonomously explore directory trees, read file contents, propose diffs, and execute verified terminal commands within strictly bounded sandbox policies. Further Research Directive: Research Anthropic Computer Use tool integration for headless browser interaction and visual user interface regression testing.

ATOM AI3: CURSOR AI COMPOSER MULTI-FILE SYNTHESIS ENGINE File Path: src/main/kotlin/atropos/core/agent/AgentEditProtocol.kt Callers and Interfaces: AgentEditDecoder and AgentEditMaterializer classes, called by AgentPatchStore and AutonomousWaveOrchestrator. Values and Symbols: AgentEditOperation, Create, Rewrite, Replace, AgentEditMaterialization, materialize, renderDiff. Imports and Exports: Imports java.nio.file.Path, java.security.MessageDigest; exports atomic multi-file patch materializations. Implementation Directive: Support natural language multi-file code composition. Parse AI responses containing multiple file rewrite and replace blocks into an atomic unified diff, validating target file existence and cryptographic hashes before applying edits. Further Research Directive: Research syntactic AST merge strategies to apply AI code suggestions without disturbing surrounding unformatted comments or formatting.

ATOM AI4: CONTINUE DEV OPEN-SOURCE LOCAL MODEL CONNECTOR File Path: src/main/kotlin/atropos/core/integration/McpCatalogConfigHarness.kt Callers and Interfaces: McpCatalogConfigHarness class, called by McpHostManager and SettingsMatrixPage. Values and Symbols: McpCatalogEntry, validateServerPrerequisites, generateVerifiedMcpJson, localOnlyMode, openSourceModelRegistry. Imports and Exports: Imports org.json.JSONObject; exports verified configuration manifests for open-source AI servers. Implementation Directive: Connect local IDE workspaces to self-hosted open-source large language models (such as DeepSeek, Qwen, or Mistral). Enforce an air-gapped local-only mode that guarantees no code or prompts egress beyond the local subnet. Further Research Directive: Research llama.cpp server and vLLM OpenAI-compatible endpoint optimizations for low-latency speculative decoding on local workstations.

ATOM AI5: CODY ENTERPRISE REPOSITORY GRAPH NAVIGATOR

File Path: src/main/kotlin/atropos/core/supremacy/CausalImpactGraphEngine.kt Callers and Interfaces: CausalImpactGraphEngine class, called by AgentContextCollector and DecoupledWorkerSynthesizer. Values and Symbols: computeTransitiveClosure, SymbolRef, CausalImpactReport, packagePattern, importPattern. Imports and Exports: Imports java.nio.file.Path; exports transitive dependency closure graphs across repositories. Implementation Directive: Leverage multi-repository AST symbol graphs to answer deep structural and architectural questions. Trace call hierarchies across package boundaries and map transitive dependency impacts before approving code refactorings. Further Research Directive: Research SCIP (Source Code Intelligence Protocol) index generation for instantaneous cross-repository symbol jump navigation.

ATOM AI6: TABNINE PERMISSIVE-LICENSE CODE COMPLETION ENGINE File Path: apps/web/src/components/ide/monaco-code-editor.tsx Callers and Interfaces: MonacoCodeEditor inline completion provider, invoked by editor keystroke listeners. Values and Symbols: tabnineAutocompleteProvider, provideInlineCompletions, trainedOnPermissiveOnly boolean, attributionCert. Imports and Exports: Imports monaco inline completion types; exports predictive ghost-text suggestions. Implementation Directive: Provide predictive inline ghost-text code completions using neural models trained strictly on permissive open-source licenses (MIT, Apache 2.0). Emit cryptographic attestation certificates proving non-infringing code origin. Further Research Directive: Research on-device quantized small language models (sub-1B parameters) running in WebAssembly for zero-latency offline autocompletion.

ATOM AI7: CODEIUM UNLIMITED INLINE COMPLETION ROUTER File Path: src/main/kotlin/atropos/core/provider/FreeProviderCapabilityRanker.kt Callers and Interfaces: FreeProviderCapabilityRanker object, called by ProviderCascadeRouter and RoutePolicy. Values and Symbols: capabilityScore, isFree, rankFreeFirst, ProviderDescriptor, ApiCapability.CODE. Imports and Exports: Imports atropos.core.provider.ProviderDescriptor; exports prioritized free provider rankings. Implementation Directive: Route code generation, inline autocompletion, and technical chat queries to high-throughput free-tier inference gateways. Monitor response quality and automatically fail over to secondary providers upon encountering rate limits. Further Research Directive: Research token-bucket rate limit scheduling heuristics that pace completion requests to stay precisely beneath provider quota thresholds.

ATOM AI8: SUPERMAVEN MASSIVE-CONTEXT PREDICTIVE ENGINE File Path: src/main/kotlin/atropos/core/supremacy/ContextBudgetEngine.kt Callers and Interfaces: ContextBudgetEngine class, called by CodebaseContextPacker and DynamicIssueRouter. Values and Symbols: filterAndAllocate, calculateShannonEntropy, ContextChunk, AllocatedContext, contextWindowTokens. Imports and Exports: Imports standard collections; exports densely allocated context payloads. Implementation Directive: Ingest massive multi-million-token codebase contexts to fuel high-speed predictive code completion.

Prune redundant AST nodes using Shannon entropy bounding to fit maximum relevant repository context into active prompt windows. Further Research Directive: Research Byte-Pair Encoding (BPE) token compression techniques tailored to structured programming languages to increase effective context density.

ATOM AI9: PHIND SEARCH-OPTIMIZED DEVELOPER REASONER File Path: src/main/kotlin/atropos/core/provider/adapter/DataInfraResearchProviderCatalog.kt Callers and Interfaces: DataInfraResearchProviderCatalog object, called by SerpApiKernelAdapter and ResearchOrchestrator. Values and Symbols: DataInfraProviderSpec, parseSearchResult, searchEndpoint, searchApiKey, extractWebSnippets. Imports and Exports: Imports atropos.core.provider.ApiCapability; exports search-enhanced developer research specs. Implementation Directive: Connect the editor to search-optimized developer reasoning models. Provide live web grounding to resolve documentation queries, troubleshoot obscure error codes, and verify framework API changes against the live web. Further Research Directive: Research automated web citation verification using cryptographic SHA-256 snapshots of referenced documentation URLs.

ATOM AI10: LOCAL LLAMA HARDWARE OLLAMA INTEGRATION File Path: src/main/kotlin/atropos/core/ProviderState.kt Callers and Interfaces: OllamaHealthProbe class, invoked by OllamaProvider and BackendDoctor. Values and Symbols: OllamaHealthProbe, probe, defaultHost, OllamaStatus, isReachable, loadedModels. Imports and Exports: Imports java.net.http.HttpClient; exports local Ollama daemon operational status. Implementation Directive: Connect the IDE directly to local hardware inference via Ollama or llama.cpp. Support running local models (Llama 3, DeepSeek-Coder) for completely private, offline code completion and chat with zero network leakage. Further Research Directive: Research Vulkan and Metal GPU compute backend auto-tuning in local model runners to maximize token-generation speed on consumer laptops and mobile SoCs.

ATOM AI11: EMBEDDED SQLITE-VEC WORKSPACE RAG RETRIEVAL File Path: src/main/kotlin/atropos/core/memory/SqliteVecMemoryIndex.kt Callers and Interfaces: SqliteVecMemoryIndex class, invoked by LocalMemoryStore and DecoupledWorkerSynthesizer. Values and Symbols: index, search, VectorHit, IndexResult, stableRowId, sqlVector, cosineDistance. Imports and Exports: Imports atropos.core.policy.BoundedProcessRunner; exports semantic similarity search hits. Implementation Directive: Chunk and embed workspace files into an embedded sqlite-vec database. Enable semantic retrieval-augmented generation (RAG) to allow autonomous agents to retrieve relevant codebase snippets based on concept similarity. Further Research Directive: Research hybrid dense-sparse retrieval combining sqlite-vec cosine distance with BM25 full-text keyword indexing for enhanced code recall accuracy.

ATOM AI12: ABSTRACT SYNTAX TREE CONTEXT MCP SERVER File Path: src/main/kotlin/atropos/core/synthesis/AstSurgicalPatcher.kt Callers and Interfaces: AstSurgicalPatcher class, called by TreeSitterGrammarBridge and AutonomousAstHealingLoop. Values and Symbols: spliceNode, Kotlin-

ParseTree, diagnosticScope, replacement, CodeFenceExtractor. Imports and Exports: Imports atropos.core.parser.KotlinParseTree; exports surgically patched syntax trees. Implementation Directive: Expose an MCP server tool that parses source code into structural AST DAGs. Allow AI models to query function boundaries, callers, type signatures, and perform surgical node replacements rather than crude regex string edits. Further Research Directive: Research CST (Concrete Syntax Tree) lossless token re-synthesis to preserve exact indentation, whitespace, and comments during AST node replacement.

ATOM AI13: REPOSITORY PROCESS TELEMETRY LOGGER MCP File Path: src/main/kotlin/atropos/core/telemetry/FrontierSweTelemetryRecord.kt Callers and Interfaces: FrontierSweTelemetryRecord companion object, invoked by TelemetryRingBuffer and TelemetrySanitizationGate. Values and Symbols: FrontierSweTelemetryRecord, TelemetryKind enum, canonicalJson, sha256Hex, ThoughtTraceProcessGraph. Imports and Exports: Imports java.security.MessageDigest, java.time.Instant; exports immutable telemetry event frames. Implementation Directive: Record human developer interactions, autonomous agent thought traces, compiler errors, and tool executions into an attested process supervision dataset. Stream sanitized logs to research repositories for continuous model improvement. Further Research Directive: Research differential privacy noise injection algorithms for process telemetry to guarantee zero leakage of proprietary business logic in training data.

ATOM AI14: TERMINAL ERROR SELF-CORRECTION FEEDBACK LOOP File Path: src/main/kotlin/atropos/core/repair/AutonomousAstHealingLoop.kt Callers and Interfaces: AutonomousAstHealingLoop class, invoked by FactoryRepairRouter and VerifiedCompletionGate. Values and Symbols: sliceCompilerLog, executeRepairIteration, verifyHealedDiff, DiagnosticSlice, HealingAttemptResult. Imports and Exports: Imports java.nio.file.Path; exports validated healing patches and verification receipts. Implementation Directive: Automatically intercept non-zero terminal exits and compiler error logs. Slice the error stack trace, locate the failing code symbol, and loop the diagnostic context back into the AI model to autonomously synthesize and verify a fix. Further Research Directive: Research multi-candidate speculative repair generation where multiple candidate patches are compiled in parallel across isolated worktrees.

ATOM AI15: AUTONOMOUS WEB SERVER MOCK AND TEST SANDBOX File Path: src/main/kotlin/atropos/core/multimodal/LivePreviewEvidenceService.kt

Callers and Interfaces: LivePreviewEvidenceService class, called by BrowserActuator and VerifiedCompletionGate. Values and Symbols: captureUrl, captureStaticHtml, BrowserEvidenceResult, BrowserEvidenceStatus, expectedText. Imports and Exports: Imports atropos.core.multimodal.BrowserEvidenceResult; exports automated browser preview receipts. Implementation Directive: Allow autonomous AI agents to spin up local test servers in an isolated sandbox, execute HTTP requests against endpoints, capture visual DOM snapshots, and mathematically verify functional application acceptance criteria. Fur-

ther Research Directive: Research headless Chromium DevTools Protocol (CDP) session pooling for running multi-viewport visual layout assertions in sub-second execution windows.